

# LightTS: Lightweight Time Series Classification with Adaptive Ensemble Distillation—Extended Version

David Campos<sup>1</sup>, Miao Zhang<sup>1,2</sup>, Bin Yang<sup>1,3</sup>, Tung Kieu<sup>1</sup>, Chenjuan Guo<sup>1,3</sup>, and Christian S. Jensen<sup>1</sup>

<sup>1</sup>Aalborg University, Denmark <sup>2</sup>Harbin Institute of Technology, Shenzhen, China <sup>3</sup>East China Normal University, China

<sup>1</sup>{dgcc,tungkvt,csj}@cs.aau.dk <sup>2</sup>zhangmiao@hit.edu.cn <sup>3</sup>{byang, cjguo}@dase.ecnu.edu.cn

## ABSTRACT

Due to the sweeping digitalization of processes, increasingly vast amounts of time series data are being produced. Accurate classification of such time series facilitates decision making in multiple domains. State-of-the-art classification accuracy is often achieved by ensemble learning where results are synthesized from multiple base models. This characteristic implies that ensemble learning needs substantial computing resources, preventing their use in resource-limited environments, such as in edge devices. To extend the applicability of ensemble learning, we propose the LightTS framework that compresses large ensembles into lightweight models while ensuring competitive accuracy. First, we propose adaptive ensemble distillation that assigns adaptive weights to different base models such that their varying classification capabilities contribute purposefully to the training of the lightweight model. Second, we propose means of identifying Pareto optimal settings w.r.t. model accuracy and model size, thus enabling users with a space budget to select the most accurate lightweight model. We report on experiments using 128 real-world time series sets and different types of base models that justify key decisions in the design of LightTS and provide evidence that LightTS is able to outperform competitors.

## 1 INTRODUCTION

With the instrumentation of processes in a broad range of settings, time series data is increasingly prevalent. As a result, the ability to analyze and create value from time series data is increasingly important. For example, accurate classification of time series is important in applications related to health, in industrial automation, in web services, and in cybersecurity [1, 18, 48].

Time series classification methods are already capable of high accuracy [44]. Most state-of-the-art methods [32, 36, 37] rely on *ensemble* learning, where multiple base models perform classification jointly. Ensemble learning is based on the wisdom of the crowd, suggesting that a joint result from the crowd is often superior to any result from a single source [2]. As shown in Figure 1(a),  $N$  base models (*BM*s), e.g., classifiers based on InceptionTime [19], Temporal Dictionary [38], or Forest Classifier [14], are combined to provide a joint result. A simple and effective combination approach that is used commonly is to assign identical weights to all base model, e.g.,  $1/N$  in Figure 1(a), such that all models contribute equally to the result of the ensemble [6, 16, 31, 59].

While ensemble based methods achieve state-of-the-art accuracy, the use of multiple base models requires significant resources for storing and executing the base models, which prevents their use in resource-limited environments, e.g., on edge devices. For example, the ongoing transformation of power grids to support sustainable energy sources, e.g., wind and solar, relies on power

electronics (PE) devices that often have limited memory and computational capabilities. It is of great interest to be able to perform time series classification on PE devices—classifying workload time series into different load levels can enable adaptive control and maintenance [58]. To enable such uses, it is important to develop lightweight models, e.g., using quantized (e.g., 4, 8, or 16 bits) parameters instead of full-precision (i.e., 32 bits) parameters. For instance, a 16-bit quantized model may use only 50% of the storage of its full-precision 32-bit counterpart. Focusing on the domain of time series classification enables demonstrating the applicability of the proposed method in real-world use cases such as PE adaptive control, in addition to enabling evaluation of the proposal. Our methodology is applicable in other domains with minor adjustments.

One approach to building such lightweight models in the setting of large ensembles is Knowledge Distillation [25]. The idea is to treat the large ensemble as a *Teacher* and then train a lightweight *Student* model to mimic the results from the teacher—cf. Figure 1(b). Though being an effective approach, two main challenges remain. **Lack of Distillation Flexibility:** An important design consideration in ensemble learning is to ensure a high level of diversity among the base models. However, existing knowledge distillation employs a single distillation step that employs the ensemble’s combined output to teach the lightweight model, causing the contribution by each base model to be fixed. When an ensemble uses equal weights to combine the results of its base models, all base models contribute equally to the lightweight model. This significantly reduces the distillation flexibility. For example, some base models may be more suitable for guiding the training of a 4-bit quantized student model, while other base models may be more suitable for a 16-bit model. Existing solutions are unable to identify and exploit such diversity in the distillation process. As an analogy, in a real-world teacher-student context, it is a good strategy to align different subject teachers, e.g., physics or math teachers, with the students who lack of the corresponding knowledge, e.g., physics or math. Thus, using the same teacher, i.e., the ensemble, to teach different quantized student models is too rigid. To enhance distillation flexibility, an approach is needed that can select different base models adaptively, disregarding some models, and can assign appropriate weights to the selected models, thereby distilling knowledge according to each student model’s level of compression.

**Lack of Support for Pareto Optimal Settings:** Existing knowledge distillation considers distillations under specific student settings [40]. For instance, for a deep learning based student model, a student setting often specifies the number of layers and the quantization bits of the parameters, e.g., 3 layers, 8 bits, and 40 filter length as shown in Figure 1(b). However, in practical settings, we often do not know how to configure such specific student settings for different devices. Instead, only storage constraints are known.

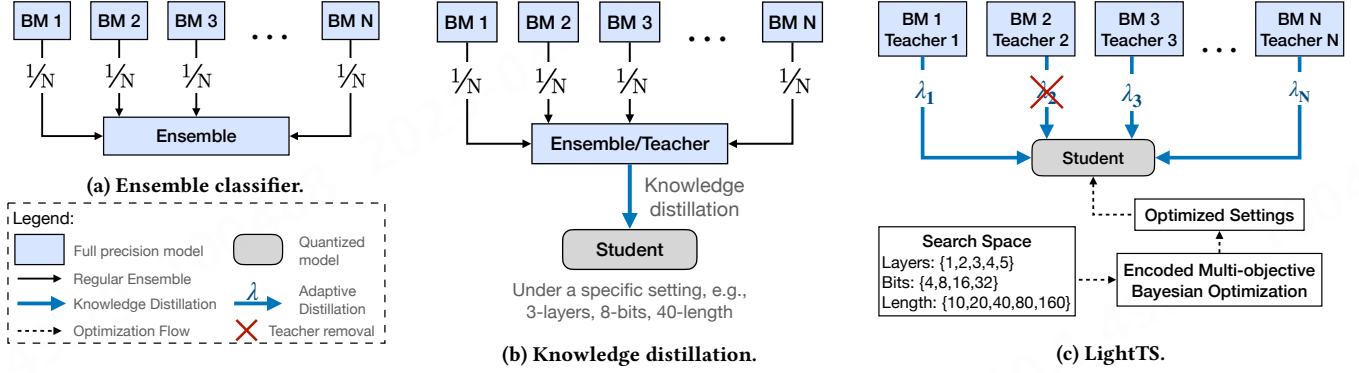


Figure 1: Ensemble Classifier, Knowledge Distillation, and LightTS.

Thus, it is beneficial to obtain Pareto optimal settings considering accuracy vs. model sizes, a.k.a. Pareto frontiers (see the example in Figure 2) over a large number of possible student settings. Given edge devices with different storage constraints, Pareto frontiers facilitate the selection of model settings with the highest accuracy while meeting the storage constraints. For example, consider Device #1 with a memory constraint of 100K. The Pareto frontier in Figure 2 implies that Model  $U$  is the optimal model for the device, as it has the highest accuracy among all models whose size is within 100K. The settings of Model  $U$ , 4 layers and 4-bit parameters, should be used for Device #1. Similarly, for Device #2 with a memory constraint of 140K, we can use the frontier to identify an optimal model, i.e., Model  $V$ . Thus, an effective and efficient method to identify the Pareto frontier from a large number of student settings is called for since the search space can reach sizes of  $10^5$  or even more.

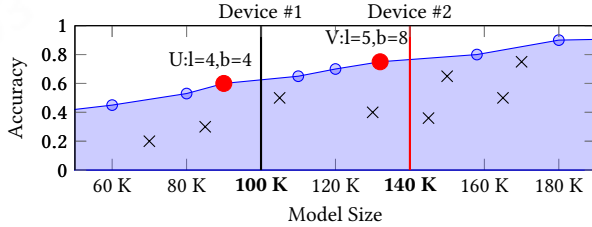


Figure 2: Pareto Frontier Example. Circles and crosses represent possible student settings. The Pareto frontier consists of circles that represent optimal settings, as no other settings exist with higher accuracy and smaller model size.

To address the two limitations, we propose LightTS, a flexible framework to obtain Lightweight Time Series classification models using novel adaptive ensemble distillation, as illustrated in Figure 1(c). LightTS is flexible in that it is model-agnostic—different types of base models, including both deep and non-deep learning, can be used in an ensemble that serves as input to LightTS.

**Addressing Challenge 1:** To ensure high distillation flexibility, we propose a novel *adaptive ensemble distillation*. LightTS treats all base models as teachers, rather than using the ensemble as a single teacher, as done in classic knowledge distillation. In addition, LightTS adaptively assigns appropriate weights to different

teachers ( $\lambda_i$  for teacher  $i$  in Figure 1(c)). Given a specific quantized student model, e.g., a 16-bit model, LightTS enables assignment of higher weights to the teachers that may contribute more knowledge to the training of the 16-bit model. It is also possible to disregard unimportant teachers by assigning them zero weight. When a differently quantized student model is called for, LightTS is able to assign a different set of weights to the teachers. This offers considerable distillation flexibility. We achieve this by a novel bi-level optimization approach—an inner level optimization learns the quantized model parameters of the student model, while an outer level optimization adjusts the teacher weights  $\{\lambda_i\}_{i=1}^N$ .

**Addressing Challenge 2:** We propose a novel encoded multi-objective Bayesian optimization method to identify the Pareto frontiers (see the lower part in Figure 1(c)). To do so, we first define a search space that models a wide variety of quantized student model settings. As shown in Figure 1(c), the search space specifies possible layers per block (e.g., 1, 2, 3, 4, or 5), quantized parameter bit-widths (e.g., 4, 8, 16, or 32 bits), and filter length (e.g., 10, 20, 40, 80, or 160). Each layer can use parameters of a distinct size, thus giving rise to a wide variety of quantized model settings.

Given a quantized model setting, we are able to obtain its accuracy, by using the proposed adaptive ensemble distillation, along with its model size, by counting the total bits. However, it is prohibitively inefficient to compute the accuracy and size of all quantized student model settings in the search space. Instead, we propose a novel encoded multi-objective Bayesian optimization method to explore the most promising settings while considering both accuracy and model size. In particular, we propose a novel encoding scheme for the search space such that the encoded space captures appropriate semantics and fits Bayesian optimization better. This enables effective identification of Pareto frontiers.

**Contributions:** To the best of our knowledge, this is the first study that introduces an adaptive ensemble distillation scheme and a generic framework to identify a set of Pareto optimal lightweight models that comply with different storage constraints for time series classification. The paper makes the following contributions.

- It proposes an ensemble distillation strategy that is able to adaptively select important base models and assign these appropriate weights, while disregarding unimportant base models, thus enabling more flexible knowledge distillation to lightweight models than hitherto possible.

- It proposes a novel encoding scheme along with an encoded multi-objective Bayesian optimization method to find Pareto frontiers, facilitating the identification of optimal models under different storage constraints.
- It reports on comprehensive experiments that justify key design decisions and demonstrate that LightTS is able to outperform state-of-the-art solutions.

The remainder of the paper is organized as follows. Section 2 covers preliminaries. Section 3 details the framework, and Section 4 reports on the experiments. Section 5 reviews related work, and Section 6 concludes.

## 2 PRELIMINARIES

This section presents concepts that are necessary to introduce the proposed framework.

### 2.1 Time Series Classification

**2.1.1 Time Series.** A time series  $\mathcal{T} = \langle t_1, t_2, \dots, t_C \rangle$  is a sequence of  $C$  observations where each observation is an  $M$ -dimensional vector, so  $t_j \in \mathbb{R}^M$ .

**2.1.2 Labeled Time Series.** A labeled time series set  $\mathcal{D}$  is a collection of  $(\mathcal{T}_i, l_i)$  pairs, where  $\mathcal{T}_i$  is a time series and  $l_i \in \mathbb{L}$  is the time series’s label, indicating a specific class among a set of classes  $\mathbb{L}$ . For example, in a human activity time series data set,  $\mathbb{L}$  includes labels representing different activity classes, such as walking, jumping, and jogging.

**2.1.3 Time Series Classification.** A time series classifier is a function that takes as input a time series and returns its corresponding label. During training, we are given a labeled time series set to train the classifier. The accuracy of the classifier is evaluated on time series that do not appear in the labeled time series set using for training.

### 2.2 The InceptionTime Classifier

We use InceptionTime [19], a state-of-the-art neural classifier for time series, as the student model in LightTS. Two considerations lead to this choice of a neural classifier for achieving lightweight student models. First, neural classifiers achieve state-of-the-art time series classification accuracy [19]. Second, neural classifiers offer a high degree of compressibility to be discussed in Section 2.3.

InceptionTime is a convolution neural network that employs variable-length convolution filters to capture temporal patterns of different time spans. Figure 3 shows an InceptionTime model with 3 blocks, each with multiple convolutional layers (cf. the zoom-in of the first block). A layer uses a specific number of same-length convolution filters to extract features from the time series  $\mathcal{T}$ . Then, the different convolution layers are stacked. Specifically,  $\mathcal{T}^{(i)} = \parallel_k \mathcal{T}^{(i-1)} * F_k$  where  $F$  is a convolution filter,  $*$  denotes 1D convolution,  $\parallel$  denotes concatenation, and  $\mathcal{T}^{(i)}$  is the output of the  $i$ -th block and  $\mathcal{T}^{(0)} = \mathcal{T}$ . The lengths of convolution filters in different layers generally vary, enabling the capture of patterns of varying lengths. More specifically, the filter lengths are decreased by half. For the 3-layer block, the filter lengths may be 40, 20, and 10. Processing time series with multiple layers with different filters provides a comprehensive context for classifying time series

since patterns of different length will fit better on the filters with the closer length, as their receptive field [33] matches the pattern. Thus, the concatenated output of the layers is the input for the following block. Finally, the output of the last block is passed to a fully connected (FC) layer with a Softmax function to assign a class distribution to the time series.

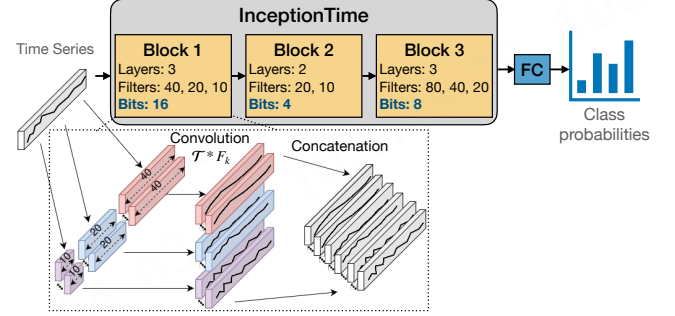


Figure 3: InceptionTime Classifier.

### 2.3 Quantized Neural Classifiers

Neural classifiers offer a high degree of compressibility. A neural classifier like InceptionTime often consists of blocks, each with multiple layers, each of which in turn includes convolutional filters with different lengths. The parameters in the filters are often 32-bit float, so full-precision InceptionTime always use 32-bit parameters. Thus, to compress a neural classifier, it is possible to reduce the number of blocks, the number of layers in a block, the filter lengths, and the bit-width of the parameters in the different layers (i.e., parameter quantization), as shown in Figure 3. For example, Figure 4 illustrates how full-precision parameters, i.e., 32-bit floats, can be quantized into 3-bit parameters using uniform quantization [23]. As 8.623728 falls into the interval [7.5, 12.5), it maps to 10, which is then mapped to the 3-bit bucket 101.

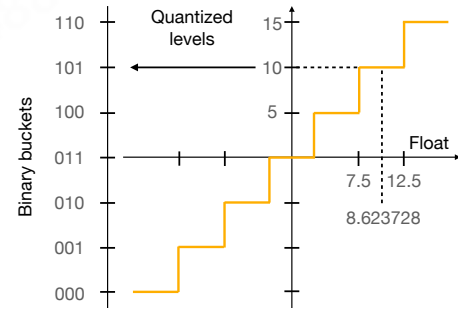


Figure 4: Quantization Mapping.

### 2.4 Knowledge Distillation

Knowledge Distillation (KD) [25] aims to transfer knowledge from a teacher model to a student model, where the teacher is often a larger model with higher discriminative capacity than the student. In classification, the knowledge is represented by a probability

distribution over classes produced by the teacher model. Let  $q$  and  $p$  represent the class distributions from the teacher and the student, respectively. Then, knowledge distillation is formalized in Equation 1, where  $\alpha \in [0, 1]$  is a hyper-parameter [54].

$$\mathcal{L} = \alpha \times \mathcal{L}_{CE}(p, y) + (1 - \alpha) \times \text{Dist}(q, p) \quad (1)$$

Specifically, the loss function is computed over two components that are weighted by  $\alpha$ . The first component is the cross-entropy (CE) between the student class probabilities  $p$  and the ground truth label  $y$ , which provides supervision from the ground truth labels. The second component represents the distance between the teacher and student distributions  $q$  and  $p$ , e.g., Kullback–Leibler (KL) divergence, to encourage a student to mimic the behavior of a more powerful teacher. They both contribute to training an accurate student.

When a group of base models is available as the teachers, their average-ensemble,  $q = 1/N \times \sum q_i$ , where  $q_i$  is the class distribution returned by the  $i$ -th base model, is typically considered as the knowledge source [17]. Figure 5 shows an example of the knowledge distillation with an ensemble consisting of three base models.

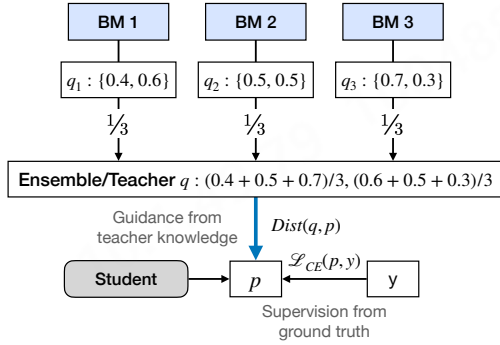


Figure 5: Classic Ensemble Knowledge Distillation.

### 3 LightTS

We first introduce the two problem scenarios supported by LightTS. Then, we proceed to introduce two main building blocks of LightTS, *adaptive ensemble distillation* and *Pareto frontier identification*, which each targets one of the two problem scenarios.

#### 3.1 Problem Scenarios

The input to LightTS is an already trained ensemble consisting of  $N$  full-precision base models  $\{BM_i\}_{i=1}^N$ , as shown in Figure 6. Although we employ InceptionTime as the quantized student model, LightTS is not limited to InceptionTime but can also support other bases models, including both deep and non-deep learning models. It is only required that the base models output class distributions. This design makes LightTS a generic framework.

The two problem scenarios supported by LightTS are shown in Figure 6. First, a specific lightweight student setting is given, e.g., the number of layers, filters length, and the quantization bits per layer. Here, the goal of LightTS is to build an accurate student model under this lightweight setting. We propose *adaptive ensemble distillation* to support this scenario (cf. Section 3.2).

However, establishing student settings that achieve good accuracy vs. space trade-offs is non-trivial, and it is inefficient to manually identify such promising settings. Therefore, we consider a second problem scenario, where the setting for the quantized student model is not given. Instead, we define a search space that covers a wide variety of lightweight student settings. Here, the goal of LightTS is to identify the Pareto frontier in this space, which includes Pareto optimal settings with the property that no other settings have higher accuracy and smaller model size. We propose an encoded multi-objective Bayes optimization method to identify the Pareto frontier (cf. Section 3.3). Then, given a device with a specific memory constraint, we can choose the setting that achieves the highest accuracy while complying with the memory constraint.

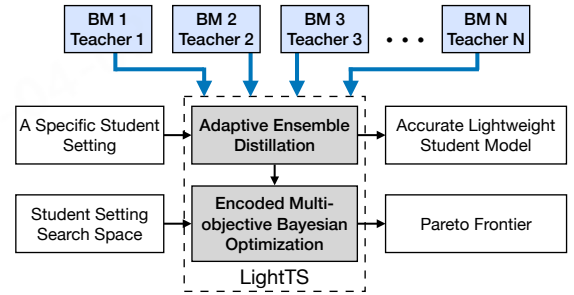


Figure 6: LightTS Workflow.

#### 3.2 Adaptive Ensemble Distillation

In the first problem scenario where a lightweight student setting is given, we propose an *adaptive ensemble distillation* (AED) process which assigns the teachers with learnable, adaptive weights, such that different teachers can contribute differently to the student. The scenario is aligned with the real-world analogy of taking advantage from the skills of particular teachers to prepare the students who require them the most, such as focusing on a specific subject teacher.

**3.2.1 Learning adaptive weights.** To obtain an accurate lightweight student, we need to consider two perspectives. First, we need the supervision from the ground truth labels in the training data. Second, we need the guidance from the knowledge of the already trained full-precision teachers. To distill knowledge from an ensemble of already trained full-precision teachers, the classic knowledge distillation (cf. Figure 5 in Section 2.4) utilizes the average knowledge of the  $N$  base models. Instead, in AED, we propose to introduce direct connections from every base model to the student, such that each base model works as a teacher, as shown in Figure 7. In the upper part, the knowledge distillation is drawn by a direct link between every teacher and the student. The knowledge from each teacher, i.e., its class distribution  $q_i$ , guides the learning of the student’s class distribution  $p_w$ , through a distance metric  $\text{Dist}(q_i, p_w)$ , controlled by a weight  $\lambda_i$ . In the bottom, the student’s class distribution  $p_w$  is also adjusted with respect to the ground truth  $y$  using the cross-entropy (CE) loss. Here, we denote the student’s class distribution by  $p_w$ , meaning that it is the distribution returned by the student with model parameter  $w$ .

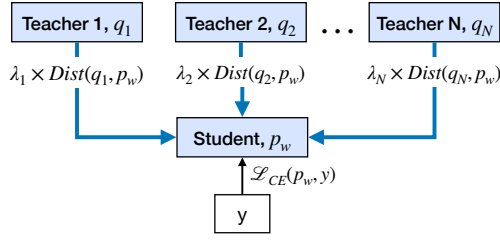


Figure 7: Adaptive Ensemble Distillation (AED).

Formally, in contrast to the ensemble distillation loss shown in Equation 1, now the distance among every teacher and the student is included as the second component weighted by their corresponding  $\lambda_i \in [0, 1]$ , where  $\sum \lambda_i = 1$ , as shown in Equation 2. Thus, instead of having one consolidated guidance between the ensemble and the student, we now consider individual guidances between the teachers and the student weighted by their associated weight  $\lambda_i$ .

$$\mathcal{L} = \alpha \times \mathcal{L}_{CE}(p_w, y) + (1 - \alpha) \times \sum_{i=1}^N \lambda_i \times \text{Dist}(q_i, p_w) \quad (2)$$

As it can be observed in the new loss function, the two components represent different types of weights that we want to optimize. First, the classification cross-entropy loss  $\mathcal{L}_{CE}(p_w, y)$  involves  $p_w$ , which is associated with quantized model parameters  $w$  from the student, e.g., quantized convolutional filters. Second, the degree of contribution  $\lambda_i$  comes from each teacher. Both components are dependent to each other because (i) when  $w$  is learned, it requires to know how much the teachers are contributing via their  $\lambda_i$ ; and (ii) while adjusting the set  $\lambda = \{\lambda_i\}_{i=1}^N$ , it requires to perceive the student accuracy, i.e., depending on  $w$ . The condition of having two optimization objectives leads us to consider a bi-level optimization modeling, where each objective can be adjusted alternatively while considering the results from each other.

In the inner-level optimization stage, we learn the model parameters  $w$  for the student. While adjusting  $w$ , the  $\lambda$  is kept static, the distances with respect to the teachers  $\text{Dist}(q_i, p_w)$  and the classification loss  $\mathcal{L}_{CE}(p_w, y)$  can change, as they are dependent on  $w$ . This step uses the training data set. In the outer-level optimization stage, the  $\lambda$  is adjusted using the validation set, while the distances w.r.t. the teachers and the cross-entropy loss remain unchangeable. Formally, the above bi-level optimization is detailed next in Equations (3) and (4), where  $D_{train}$  and  $D_{validation}$  denote the data for training and validation, respectively. It is detailed in Algorithm 1.

$$\arg \min_{\lambda} \mathcal{L}(\lambda, w^*, D_{validation}) \quad (3)$$

$$s.t., w^* = \arg \min_w \mathcal{L}(\lambda, w, D_{train}) \quad (4)$$

Equation 4 represents the inner-level optimization. Under a specific disposition of  $\lambda$ , it finds the optimized student weights  $w^*$ , meaning that under this specific  $\lambda$ , the student with  $w^*$  gives the best accuracy on the training data. It uses the training set, since in this step it is where the model is supervised trained in terms of classification performance, where cross-entropy loss with ground truths is used. The step is shown in line 6 in Algorithm 1. Here,  $p_w$  represents the class distribution for the student, given its parameters  $w$ . We

denote  $\lambda_i$  with a box to indicate that it is not changing during the inner optimization step, usually referred as parameters freezing [3]. Here, the Softmax function  $\sigma(\cdot)$  assures  $\sum \sigma(\lambda_i) = 1$ .

In Equation 3, representing the outer-level optimization, the goal is to find the best  $\lambda$  given that the student has already an optimal weight  $w^*$ , so the distances with respect to all teachers are set and weighted by  $\lambda$ . The step uses a validation set to assure that its optimization is independent with respect to the  $w^*$  adjustments, and the  $\lambda$  is optimized using back-propagation (BP). The  $\lambda$  set acts as hyper-parameters to the inner step, so also using the same training set could derive on unsought conditions such as overfitting, and thus we use the validation set instead. The step is shown in line 8 in Algorithm 1. Similarly to line 6, we use boxes to indicate that the classification cross entropy loss and the distances w.r.t. the teachers are fixed during the outer optimization. We run multiple inner-level steps for each outer-level one to have a stable training, meaning that the model will not get different  $\lambda$  values at every iteration. It gets a single set of  $\lambda$ , train for  $v$  iterations, and then get another updated set for  $\lambda$ .

**Complexity:** Algorithm 1 executes  $E$  training epochs. In each, BP updates the model parameters  $w$ ; and every  $v$  epochs, BP updates  $\lambda$ . This gives cost  $E \times BP_w + E/v \times BP_\lambda$ , where  $BP_w$  and  $BP_\lambda$  are the costs for updating the parameters  $w$  and  $\lambda$  using BP, respectively. Since there are many more parameters in  $w$  than in  $\lambda$  and  $v$  is a small constant, we obtain an asymptotic complexity of  $O(E \times BP_w)$ , the same as for classic Knowledge Distillation.

---

#### Algorithm 1 Learning Adaptive Weights

---

- 1: **Input:**  $[q_1, q_2, \dots, q_N]$ :  $N$  class distributions from full precision teachers.
  - 2:  $[\lambda_1, \lambda_2, \dots, \lambda_N] \leftarrow$  Uniform weight initialization:  $1/N$
  - 3:  $v \leftarrow$  Validation steps
  - 4: Quantized weight  $w$ :  $\leftarrow$  Random initialization
  - 5: **for** training epochs  $e \leftarrow 1, 2, \dots, E$  **do**

▷ Using training data

  - 6:  $\arg \min_w \alpha \times \mathcal{L}_{CE}(p_w, y) + (1 - \alpha) \times \sum_{i=1}^N \boxed{\sigma(\lambda_i)} \times \text{Dist}(q_i, p_w)$

▷ Using validation data

  - 7: **if**  $e \bmod v = 0$  **then**

- 8:  $\arg \min_{\lambda} \alpha \times \boxed{\mathcal{L}_{CE}(p_w, y)} + (1 - \alpha) \times \sum_{i=1}^N \sigma(\lambda_i) \times \boxed{\text{Dist}(q_i, p_w)}$
- 

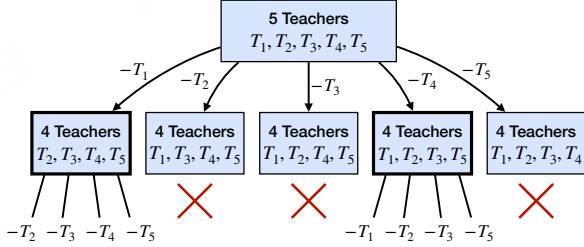
In addition to classification, the proposal can be applied to forecasting by replacing the cross entropy term in Equation 2 by a forecasting error term, e.g., mean square error.

**3.2.2 Removing Unimportant Teachers.** We have teachers and students with different strengths and limitations. Recall that a design principle of ensemble learning is to maintain high diversity among the base models, i.e., the teachers in our setting. The students are also diverse as they are constrained in different ways, e.g., different layers, filter lengths, and bit-widths. Ideally, we want to associate the students with the teachers that offer them the most knowledge (already achieved in the proposed AED), while removing teachers who are not contributing or even affecting adversely the student learning. In addition, removing teachers that do not contribute

to the student makes it easier to assign more accurate adaptive weights in future iterations.

A simple yet effective principle for removing unnecessary teachers is considering the student classification accuracy as the indicator of improvement. In the current scenario, we can use the student accuracy as the metric for evaluating if the removal of one teacher is contributing to a better guidance to the student. The criteria can be applied with the leave-one-out idea [39] that iteratively removes teachers. After removing a teacher, we can check whether the student improves the classification accuracy. For example, in Figure 8 we present a scenario with five teachers. If we want to remove one teacher, there are five different ways to do so, i.e., removing  $T_1, T_2, T_3, T_4$ , or  $T_5$ . Then, we can evaluate whether removing  $T_i$  results in student accuracy improvement. If so, we consider further removing. Otherwise, we stop removing. For example, if removing  $T_1$  or  $T_4$  further improves the student performance, while  $T_2, T_3$ , and  $T_5$  decrease it, we continue the leave-one-out removal for  $T_1$  and  $T_4$  branches, but stop further removal for the other 3 cases, as Figure 8 shows.

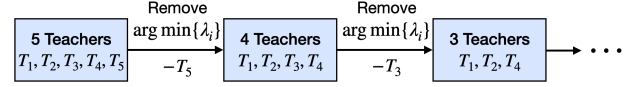
This solution is not efficient if we only rely on the student performance, because we only rely on results but not criteria to choose the teachers that will be removed. Therefore, conducting an exhaustive exploration becomes very costly, as shown in Figure 8, where leave-one-out of five teachers, marked as  $-T_i$ , at the first two iterations are exemplified. In the worst case, the number of cases to evaluate following this strategy grows at a factorial pace, so it becomes an intractable problem with relatively few cases, e.g., ten teachers require  $10! \approx 3.6 \times 10^6$  evaluations.



**Figure 8: Leave-one-out Based Teacher Removal, Yielding Factorial Growth in the Worst Case.**

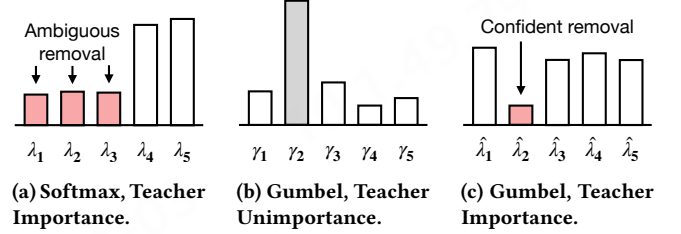
We propose to utilize the weight  $\lambda$  associated with the teachers to facilitate a more efficient teacher removal strategy. More specifically, the teacher with the lowest  $\lambda_i$  is removed. After removing the teacher, we conduct a new round of AED. We keep removing the teacher with the lowest  $\lambda$  until running out of base models. The maximum possible number of iterations is linear, i.e., removing all  $N$  teachers. Finally, we return the teacher configuration with the highest accuracy. Figure 9 shows an example where we remove  $T_5$  and then  $T_3$ .

A condition that arises over using  $\lambda$  as the indicator for removing teachers is that, after assuring  $\sum \sigma(\lambda_i) = 1$  using a Softmax function, sometimes the  $\lambda$  values become very close. Thus, it becomes ambiguous which is the weakest teacher to be removed since more than one can be suitable. An example of five teachers is shown in



**Figure 9: LightTS Teacher Removal.**

Figure 10(a), where the first three teachers have similarly small  $\lambda$  values.



**Figure 10: Confident Teacher Removal.**

To contend with the above undesired condition, we need a distribution where the majority of weights have similarly large values and only one is significantly smaller, as Figure 10(c) shows. In such a scenario, the removal decision is more confident, as a clearly weak teacher is removed.

To this end, we consider a reparameterization trick, specifically the Gumbel-Max Trick [28, 35], to introduce stochastic noises  $g_s$  into weights  $\lambda$  during the teacher removal to make the “removing” part differentiable, and leverage a temperature factor  $\tau$  to control the sharpness of the reparameterized distribution,  $\lambda_i^* = \frac{\exp((\lambda_i + g_{s_i})/\tau)}{\sum_j \exp((\lambda_j + g_{s_j})/\tau)}$ . In this way, the reparameterized weight contains a deterministic part, i.e.,  $\lambda$ , and a stochastic part, i.e.,  $g_s$ , enabling to sample a teacher to be removed instead of just taking the  $\arg\max$ , given their different probabilities. The Gumbel-Max Trick considers the Gumbel distribution [24] for noises  $g_s$  which is stable under operations that involve finding maximums [34]. More importantly, through controlling  $\tau$ , the reparameterized weights smoothly approach the discrete  $\arg\max$  computation, so as enlarging the gap between largest weight and the remaining ones and making it likely to become the maximum, as shown in Figure 10(b).

Although a Gumbel-Softmax function on  $\lambda$  enables the “removing” differentiable, it behaves similarly to the  $\arg\max$  as the temperature  $\tau$  decreases close to zero, while is contrary to our intuition which is supposed to remove the minimal one, i.e.,  $\arg\min$ . Thus, instead of reparameterizing the “importance” of each teacher, we apply the Gumbel-Softmax function to the negative of the set of  $\lambda$  to get the opposite behavior, which we call the “unimportance” of teachers  $\gamma_i = \frac{\exp((- \lambda_i + g_{s_i})/\tau)}{\sum_j \exp((- \lambda_j + g_{s_j})/\tau)}$ , which is shown in Figure 10(b). Thus, we manage to maximize the teachers unimportance, identifying the teacher that it is most likely to be disregarded based on  $\gamma$ , as shown in Figure 10(b). Then, the values are re-parameterized to teacher importance by applying a Softmax function to the negative of the unimportance  $\hat{\lambda}_i = \sigma(-\gamma_i)$ . In  $\hat{\lambda}$ , the minimal value is confidently identifiable, as Figure 10(c) shows. In Algorithm 1,  $\hat{\lambda}$  is utilized in lines 6 and 8. After Algorithm 1 finishes, the teacher with the minimal weight  $\hat{\lambda}$  is removed.

*Complexity:* Given an ensemble with  $N$  base models, i.e., teachers, we can remove at most  $N - 1$  teachers. This implies that Algorithm 1 executes at most  $N - 1$  times. Thus, the complexity of AED with teacher removal is  $O(N \times E \times BP_w)$ .

### 3.3 Identifying Pareto Frontiers

In the second problem scenario, instead of identifying a well-performing student for a specific setting, we explore a search space of settings to find optimal sets of student settings that target different space restrictions. To achieve this goal, we first define a search space that fits neural classifiers as well define the notion of the Pareto frontier. Then, we propose an effective encoding scheme, that enables use of a novel multi-objective Bayesian optimization method for identifying Pareto optimal solutions in the search space.

**3.3.1 Search Space.** The neural classifiers introduced in Section 2.3 have four components that contribute to the model size of a student: the number of blocks, the number of layers per block, the filter length, and the bit-width.

The number of blocks controls the network depth. When keeping the number of blocks fixed, changing the number of layers per block also controls the network depth. Thus, we consider a search space that includes only three dimensions: the number of layers per block, the filter length, and the bit-width per block, while keeping the number of blocks  $B$  fixed. Different filter length enables the capture of time series patterns of different time spans [19], which often vary across data sets. Thus, in addition to contributing to the model size, dynamic adjustment of the filter length enables better classification accuracy.

Given  $B$  blocks, each block may choose the number of layers from  $L = \{1, 2, 3, 4, 5\}$ , a filter length from  $F = \{10, 20, 40, 80, 160\}$ , and a bit-width from  $W = \{4, 8, 16, 32\}$ . The selected filter length applies to the first layer. In the remaining layers, the filter lengths are reduced by half. The search space then includes  $(|L| \times |W| \times |F|)^B$  different student settings, which amounts to a very large space.

A student setting  $\mathbf{x}_i$  in the search space is a sequence of  $B$  entries, corresponding to  $B$  blocks. For the  $j$ -th entry,  $1 \leq j \leq B$ , tuple  $(L_j, F_j, W_j)$  indicates that there are  $L_j$  layers, that the filter length in the first layer is  $F_j$ , and that the bit-width is  $W_j$ , as shown in Equation 5. For example, with  $B = 3$  blocks, the student setting  $\mathbf{x}_0$  has three entries, where the first entry  $(3, 20, 8)$  indicates that there are 3 layers in the first block, that the filter length of the first layer is 20, meaning that the filter lengths of the following layers are 10 and 5, and that the bit-width of filters is 8 bits.

$$\mathbf{x}_i = \begin{pmatrix} (L_1, F_1, W_1) \\ (L_2, F_2, W_2) \\ \dots \\ (L_B, F_B, W_B) \end{pmatrix}, \quad \mathbf{x}_0 = \begin{pmatrix} (3, 20, 8) \\ (4, 40, 4) \\ (2, 10, 16) \end{pmatrix} \quad (5)$$

**Applicability to Other Neural Classifiers:** Although we use InceptionTime as the base model, the proposed method is not limited to InceptionTime. With minor adjustments, the search space can be adapted to accommodate a variety of neural classifiers. More specifically, the choices of the numbers of blocks and layers and the bit-widths apply universally across different neural classifiers. The filter lengths are specific to convolutional classifiers such as

InceptionTime. When using other types of classifiers, other components need to be considered. For fully-connected classifiers [49], the number of neurons per layer needs to be included into the search space; for recurrent neural network based classifiers [30], the sizes of their recurrent weight matrices need to be included into the search space; for Transformer-based classifiers [57], the sizes of the projection matrices need to be included. Thus, the search space can be adapted easily to different classifiers. In this paper, we consider a search space based on InceptionTime because it outperforms other classifiers at time series classification [44].

**3.3.2 Pareto Frontier.** Given a student setting, we can obtain its accuracy using the proposed AED, but this is costly. Next, it is also possible to compute the size based on the setting using the information on the bits per layer, the number of filters, and filter lengths (see Section 2.3), which is very efficient. Thus, for each setting, we can define a tuple  $s$  that includes its associated accuracy and its model size—see Equation 6.

$$s_i = (\mathbf{x}_i, \text{accuracy}_i, \text{size}_i) \quad (6)$$

We use  $S$  to denote the set of all student settings. Given two settings  $s_1, s_2 \in S$ ,  $s_2$  dominates  $s_1$ , denoted as  $s_2 > s_1$ , if  $s_2$  is more accurate and not larger than  $s_1$  or if  $s_2$  is smaller than  $s_1$  and is not worse than  $s_1$  in terms of accuracy.

The Pareto frontier  $P(S)$  is a subset of  $S$ , where for every tuple  $s \in P(S)$  no other tuple  $s'$  exists that dominates  $s$ , as shown in Equation 7.

$$P(S) = \{s \in S \mid \nexists \{s' \in S \mid s' > s \wedge s' \neq s\}\} \quad (7)$$

The frontier facilitates the identification of the setting with the highest accuracy under specific model size constraints. Thus, the aim is to identify the Pareto frontier.

**3.3.3 Encoded Multi-objective Bayesian Optimization.** To get the exact Pareto frontier, it is necessary to perform domination comparisons on the  $(|L| \times |W| \times |F|)^B$  settings in the search space, e.g., using skyline querying algorithms [5]. However, since evaluating the accuracy for a setting using AED is costly, it is infeasible to evaluate the accuracy of all  $(|L| \times |W| \times |F|)^B$  settings using AED. Instead, we evaluate the accuracy of a small subset of  $Q$  ( $Q \ll (|L| \times |W| \times |F|)^B$ ) settings, to obtain an approximate frontier.

A naive way to select the  $Q$  settings is to apply random sampling. Although being simple, random sampling may not be effective as it takes into account neither accuracy nor model size. We propose instead a novel encoded multi-objective Bayesian optimization method to focus on exploring the most promising settings in the search space, by considering both accuracy and model size—see Figure 11.

To initialize the optimization process, we first randomly select  $P$  settings,  $P < Q$ . We evaluate these using AED to obtain their accuracies and also compute their sizes. They constitute the set of evaluated settings ( $ES$ ).

In classic multi-objective Bayesian optimization (MOBO), as shown in the white boxes in Figure 11, the evaluated settings  $ES$  are used to build a Gaussian Process (GP) that estimates the accuracies of the unevaluated settings. We use a GP because it is able to estimate not only the accuracy but also a probability distribution

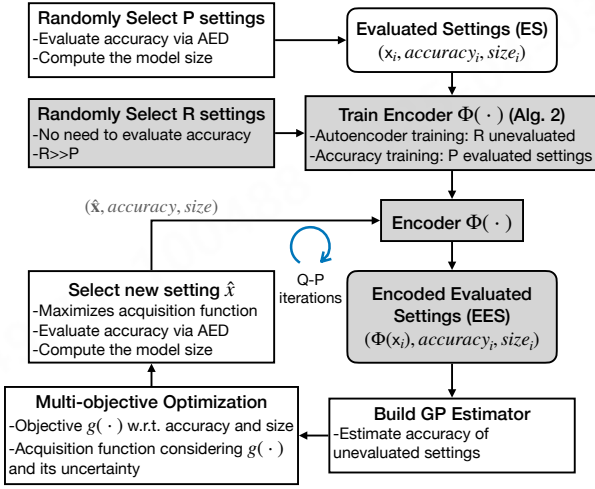


Figure 11: Encoded Multi-Objective Bayesian Optimization.

of the estimated accuracy, which is helpful for guiding the subsequent exploration. An objective function is built considering both the estimated accuracies and model sizes, and then an acquisition function helps identify the most promising setting  $\hat{x}$  based on the objective function. We evaluate  $\hat{x}$ 's accuracy using AED, compute its size, and feed it back to the GP estimator to get it updated. Then, a new iteration starts until we have evaluated  $Q$  settings.

**GP estimator:** A GP is a probabilistic regression model, that learns a mapping  $f(\cdot)$  from a multidimensional point, e.g., a student setting  $\mathbf{x}_i$ , to a real value, e.g., the accuracy  $\mu_i$  of the setting, along its probability distribution, so  $f(\mathbf{x}_i) \sim \mathcal{N}(\mu_i, \sigma_i^2)$ . Given  $P$  evaluated settings  $\mathbf{x}_{1:P}$ , the joint distribution is:

$$f(\mathbf{x}_{1:P}) \sim \mathcal{N}(\text{AED}(\mathbf{x}_{1:P}), \mathbf{K}(\mathbf{x}_{1:P}, \mathbf{x}_{1:P})), \quad (8)$$

where  $\text{AED}(\mathbf{x}_{1:P})$  is the evaluated accuracies for the  $P$  settings, and  $\mathbf{K}$  is the covariance matrix with kernel function  $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ . A generally used kernel  $\kappa$  is the squared exponential covariance function:  $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \theta_f \exp\left(-\left|\frac{(\mathbf{x}_i - \mathbf{x}_j)^2}{2\Theta^2}\right|\right)$  with variance  $\theta_f$ , scaled by the noise level  $\Theta$  of the observations.

Taking the GP as the prior [42], the posterior predictive distribution for an unevaluated setting  $\mathbf{x}^*$  is defined as:

$$f(\mathbf{x}^*) \sim \mathcal{N}(\mu(\mathbf{x}^*), \sigma^2(\mathbf{x}^*)), \quad (9)$$

where  $\mu(\mathbf{x}^*) = \kappa(\mathbf{x}^*, \mathbf{x}_{1:P}) \mathbf{K}(\mathbf{x}_{1:P}, \mathbf{x}_{1:P})^{-1} \mathbf{f}(\mathbf{x}_{1:P})$ , and  $\sigma^2(\mathbf{x}^*) = \kappa(\mathbf{x}^*, \mathbf{x}^*) - \kappa(\mathbf{x}^*, \mathbf{x}_{1:P}) \mathbf{K}(\mathbf{x}_{1:P}, \mathbf{x}_{1:P})^{-1} \kappa(\mathbf{x}^*, \mathbf{x}_{1:P})^T$ .

As the kernel function  $\kappa(\mathbf{x}_i, \mathbf{x}_j)$  suggests, the distance between two different settings affects the GP estimator significantly: if the distance between two settings is small, they are expected to have similar accuracy. Therefore, a meaningful distance metric between settings is important. We proceed to illustrate why the Euclidean distance on the space of original settings fails to be meaningful and then elaborate a novel encoding scheme such that the Euclidean distance on the encoded space offers meaningful distances.

**Complexity:** The process shown in Figure 11 requires running AED with teacher removal  $Q$  times, yielding  $O(Q \times N \times E \times BP_w)$ . In addition, the GP estimator is run  $Q - P$  times. The complexity of a

GP estimator is  $O(n^3)$  due to the matrix inverse operation, where  $n$  is the size of the covariance matrix  $\mathbf{K}$ . The size of  $\mathbf{K}$  keeps increasing as the steps continue and reaches  $Q$  in the last step. Thus, the GP part is  $O(Q^4)$ . In total, we get  $O(Q \times N \times E \times BP_w + Q^4)$ .

**Problems of the Original Space:** Consider three settings  $\mathbf{x}_1$ ,  $\mathbf{x}_2$ , and  $\mathbf{x}_3$ , all with three blocks, as shown in Equation 10. We also show their accuracies and the Euclidean distances  $\overline{\mathbf{x}_1 \mathbf{x}_2}$  and  $\overline{\mathbf{x}_1 \mathbf{x}_3}$  in the original space.

$$\mathbf{x}_1 = \begin{pmatrix} (4, 40, 8) \\ (4, 40, 8) \\ (4, 40, 8) \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} (1, 40, 8) \\ (1, 40, 8) \\ (1, 40, 8) \end{pmatrix}, \mathbf{x}_3 = \begin{pmatrix} (4, 40, 16) \\ (4, 40, 16) \\ (4, 40, 16) \end{pmatrix} \quad (10)$$

|           |          |                                               |                                                |
|-----------|----------|-----------------------------------------------|------------------------------------------------|
| Accuracy: | 0.37     | 0.24                                          | 0.38                                           |
| Distances | Original | $\overline{\mathbf{x}_1 \mathbf{x}_2} = 5.19$ | $\overline{\mathbf{x}_1 \mathbf{x}_3} = 13.85$ |
|           | Encoded  | $\overline{\mathbf{x}_1 \mathbf{x}_2} = 3.33$ | $\overline{\mathbf{x}_1 \mathbf{x}_3} = 1.70$  |

Settings  $\mathbf{x}_1$  and  $\mathbf{x}_2$  have the same bit-width, but they have different numbers of layers, 4 vs. 1, yielding quite different neural structures and dissimilar accuracies. In contrast,  $\mathbf{x}_1$  and  $\mathbf{x}_3$  differ on the bit-width, 8 vs. 16, but share the same number of layers and have similar accuracy. Based on the above,  $\overline{\mathbf{x}_1 \mathbf{x}_3}$  should be smaller than  $\overline{\mathbf{x}_1 \mathbf{x}_2}$ , as  $\mathbf{x}_1$  and  $\mathbf{x}_3$  have more similar accuracies.

However, in the original space,  $\overline{\mathbf{x}_1 \mathbf{x}_3}$  is much larger than  $\overline{\mathbf{x}_1 \mathbf{x}_2}$ , as the bit-width difference  $(16 - 8)^2$  is larger than the layer difference  $(4 - 1)^2$ . Thus, the distance in the original space is not aligned with the accuracy similarity. This happens because the values are discrete and the different semantics of the different values are also different dimensions, so adding up the distances directly is semantically incorrect.

**A Novel Encoding Scheme:** To address the problem of inconsistent distances, we propose to utilize a continuous latent space instead of the original discrete values to capture semantic similarities. In addition, a continuous space is known to fit GPs better [15, 46]. Thus, we propose an encoder  $\Phi: \mathbf{x} \rightarrow \mathbf{z}$  that maps the setting  $\mathbf{x}_i$  to a continuous feature vector  $\mathbf{z}_i$ . Then, we use the encoded  $\mathbf{z}_i$  as the input to the GP.

We proceed to cover the training of the encoder  $\Phi$ . The training involves two phases, as illustrated in Figure 12 and detailed in Algorithm 2. In the first phase, we connect the encoder with a decoder  $\Gamma(\cdot)$ . To train the encoder and decoder jointly, we random sample  $R$  unevaluated settings, without knowing the accuracies of the  $R$  settings, where  $R \gg P$ . We use the decoder to reconstruct the  $R$  settings  $\{\mathbf{x}_r\}_{r=1}^R$ . It computes  $\mathbf{x}'_r = \Gamma(\Phi(\mathbf{x}_r))$  and then minimizes the loss  $\mathcal{L}_{recons}(\mathbf{x}_r, \mathbf{x}'_r) = \frac{1}{R} \sum_{r=1}^R (\mathbf{x}_r - \mathbf{x}'_r)^2$ . In other words, the encoder and decoder work together as an autoencoder. This enables an encoder that is able to transform a discrete setting  $\mathbf{x}_r$  into a continuous vector  $\mathbf{z}_r$ . However, since the training data remains "accuracy blind," the encoded continuous space fails to capture the semantics related to accuracy. In the second phase, we therefore connect the encoder with an accuracy predictor  $\Psi(\cdot)$  and train the encoder and predictor jointly using the  $P$  evaluated settings  $\{\mathbf{x}_p\}_{p=1}^P$ , so the predictor estimates  $\text{accuracy}_p' = \Psi(\Phi(\mathbf{x}_p))$  by minimizing the loss  $\mathcal{L}_{accur}(\text{accuracy}_p, \text{accuracy}_p') = \frac{1}{P} \sum_{p=1}^P (\text{accuracy}_p - \text{accuracy}_p')^2$ . Since the encoder is already trained with the decoder using  $R$  unevaluated settings, we use an only small amount of  $P$ , where  $P \ll R$ , evaluated settings to fine tune it, such that the encoded space is

aligned better with accuracy. As Algorithm 2 shows, we use the two training phases interchangeably.

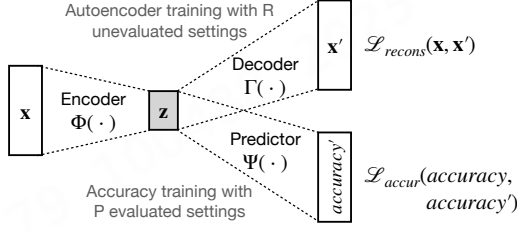


Figure 12: Two-phase Training for Encoder  $\Phi(\cdot)$ .

---

**Algorithm 2** Two-phase Encoder  $\Phi(\cdot)$  Training.

---

```

1: Input: R unevaluated settings:  $\{\mathbf{x}_r\}$ , P evaluated settings:
   ( $\mathbf{x}_i$ ,  $accuracy_i$ )
2: Output: Encoder  $\Phi$ 
3:  $ps \leftarrow$  Predictor steps
4: Encoder  $\Phi$ , Decoder  $\Gamma$ , Predictor  $\Psi$ 
5: for  $e \leftarrow 1, 2, \dots, Epochs$  do
6:    $\mathbf{x}'_r \leftarrow \Gamma(\Phi(\mathbf{x}_r))$  ▷ Autoencoder training, using R
7:    $\arg \min_{\Phi, \Gamma} \mathcal{L}_{recons}(\mathbf{x}_r, \mathbf{x}'_r)$ 
8:   if  $e \bmod ps = 0$  then ▷ Predictor training, using P
9:      $accuracy'_p \leftarrow \Psi(\Phi(\mathbf{x}_p))$ 
10:     $\arg \min_{\Phi, \Psi} \mathcal{L}_{accr}(accuracy_p, accuracy'_p)$ 

```

---

The integration of the encoding scheme into the overall Bayesian Optimization process is shown in Figure 11 using grey boxes. After training the Encoder  $\Phi(\cdot)$  using  $R$  unevaluated settings and  $P$  evaluated settings, it is used to map all the evaluated settings ( $ES$ ) to the corresponding encoded space, getting the Encoded Evaluated Settings ( $EES$ ). Then, based on the  $EES$ , we build a GP to estimate the accuracy of the unevaluated settings. To facilitate the identification of settings on the Pareto frontier, given an unevaluated setting  $\mathbf{x}^*$ , we define a joint objective function  $g(\mathbf{x}^*) = \beta \times f(\mathbf{x}^*) - (1 - \beta) \times Size(\mathbf{x}^*)$  that considers both the estimated accuracy  $f(\mathbf{x}^*)$  and the computed model size  $Size(\mathbf{x}^*)$ , preventing divergent optimizations if they are considered separately. We use different  $\beta$  in different optimization iterations, facilitating searches with different trade-offs between accuracy and model size. More specifically,  $\beta$  is randomly sampled following the PACE model strategy [29]. Since the estimated accuracy  $f(\mathbf{x}^*)$  is uncertain, the joint objective  $g(\mathbf{x}^*)$  is also uncertain. Next, we use the Expected Improvement ( $EI$ ) acquisition function [10, 11] based on the joint objective  $g(\cdot)$ , to determine the most promising setting  $\hat{\mathbf{x}}$ . We evaluate  $\hat{\mathbf{x}}$ 's accuracy using AED and compute its size, and we use the trained encoder  $\Phi(\cdot)$  to encode  $\hat{\mathbf{x}}$  and then add it to  $EES$ . Based on the updated  $EES$ , the GP estimator is also updated. Then, a new iteration starts.

## 4 EXPERIMENTS

### 4.1 Experimental Setup

**4.1.1 Data Sets.** We use the UCR time series archive [9] that comprises an extensive catalogue of time series from different domains.

Experiments are conducted on the 128 available data sets, and we focus on data sets with many classes since they are the most challenging and have important real-world uses. Details of the selected subset of the data sets are shown in Table 1, including the number of classes, the sizes of the training, validation, and testing sets, the domains, and the average time series lengths.

Table 1: Data Sets.

| Data set   | Classes | Train/Val/Test  | Domain     | Avg. Len. |
|------------|---------|-----------------|------------|-----------|
| Adiac      | 37      | 312/78/391      | Images     | 176       |
| Crop       | 27      | 5720/1440/16800 | Images     | 46        |
| FaceAll    | 14      | 448/112/1690    | Images     | 131       |
| NonInvECG1 | 42      | 1440/360/1965   | ECG        | 750       |
| NonInvECG2 | 42      | 1440/360/1965   | ECG        | 750       |
| Phoneme    | 39      | 171/43/1896     | Sound      | 1024      |
| PigAirway  | 52      | 83/19/208       | Blood flow | 2000      |
| PigArt     | 52      | 83/19/208       | Blood flow | 2000      |
| UWave      | 8       | 1680/560/2241   | Motion     | 315       |

**4.1.2 Evaluation Metrics.** To assess performance, we consider several metrics. First, *Accuracy* is the percentage of cases where the class with the highest output probability has the correct class label. It is also used in conjunction with the null-hypothesis Friedman test [20] and the Wilcoxon-Holm post-hoc method [27, 50] to evaluate all data sets. Next, *Top-5 Accuracy* evaluates the classification results based on the top-5 output probabilities. If the probability associated with the correct label is within the top-5, the result is considered as correct. It is used in the experiments involving data sets with many labels. Finally, *Model size* indicates how much memory a model requires for a specific configuration of layers and bit-width.

**4.1.3 Baselines.** LightTS is compared to four knowledge distillation methods. Although these methods use different strategies to combine the results from base models into a single teacher model, they share the same distillation scheme—the distillation is conducted between a single teacher and a student. (1) In Classic Knowledge Distillation (Classic KD) [25, 52], the teacher is the average of the class probabilities across all base models, as shown in Equation 1 in Section 2.4. The next three baselines aims at improving Classic KD by proposing different techniques to weigh the base models differently, i.e., finding values other than the  $1/N$  in Figure 1(b). (2) Adaptive Ensemble (AE-KD) [17] optimizes the weights using Support Vector Machines according to the teachers diversity derived from gradient changes. (3) Reinforced Multi-Teacher (Reinforced) [54] computes the weights following a reinforced learning process using the training losses as the reward metric. (4) Cross-validation Accuracy Weighted Probabilistic Ensemble (CAWPE) [31] uses the cross-validation accuracy results to assign the weights. Thus, if a model has a high accuracy during validation, it has a high weight in the final ensemble. In addition, we include two variations of LightTS. (5) AED-L00: we use AED with leave-one-out teacher removal. (6) AED-One: we use AED without teacher removal, i.e., Algorithm 1 is run only once. Finally, we also include the full-precision ensemble FP-Ensem for reference. As knowledge distillation can boost student performance [45, 53], a quantized student can possibly outperform FP-Ensem.

**4.1.4 Ensemble Teachers.** To evaluate the generality of LightTS on different types of ensembles, we consider ensembles with a wide variety of base models. The ensembles for all cases have  $N = 10$  base models, and the base models are initialized with different random states to ensure diversity. The base models used are detailed as follows. (1) InceptionTime [19] is the state-of-the-art deep learning method [44]. It is the default base model in the experiments. We consider three types of non-deep learning based methods that are components for state-of-the-art meta-ensemble classifiers [37]. (2) Temporal Dictionary Ensemble (TDE) [38] transforms a time series into a bag of segments of a given size and discretizes them as words. Then, it draws a histogram for the word counting and applying a nearest neighbor algorithm to classify the transformed series. (3) Canonical Interval Forest Classifier (CIF) [36] builds a Time Series Forest using a set of 22 particular features [32] to summarize them in intervals that are used to classify the time series. (4) Time Series Forest Classifier (Forest) [14] builds several trees for representing a time series, summarizing them in intervals. Then, a forest is built over the trees to identify singular features in the intervals to perform the classification of the series.

**4.1.5 Implementation Details.** The LightTS framework is implemented using Python 3.9.7, the machine learning architecture PyTorch 1.9.1, and the Bayesian Optimization infrastructure of BoTorch 0.5.1. The source code is publicly available at <https://github.com/d-gcc/Distiller>. All the models are tested using Titan RTX GPUs with 24GB of VRAM under Ubuntu 20.04.3 on an Intel Xeon W-2155 with 128GB of RAM.

To ensure fair comparisons, we follow common machine learning practices and use the validation set to adjust the hyper-parameters for all methods. The ensemble of ten base teacher models is trained in full-precision using different random seeds to ensure diversity. They are trained for 1,500 epochs, with a learning rate of 0.01, Adam optimized, and using a batch size of 64. Then, following the same configuration, the distilled student is quantized according to the testing configuration, uses a validation set of 20% with Stochastic Gradient Descent as the optimizer, and the weights  $\lambda$  are adjusted every  $v = 50$  epochs. The predictor for the encoded MOBO is also adjusted every 50 epochs.

## 4.2 Experimental Results

**4.2.1 Problem Scenario 1.** We consider lightweight students with 3 blocks and 3 layers per block. All layers use the same bit-width, chosen among 4-bit, 8-bit, and 16-bit. This means that the lightweight students obtained by different methods have the same size. Thus, we only evaluate their accuracy.

**InceptionTime as Base Models:** We report the results for the complete UCR data sets using InceptionTime as the base models. Figure 13 shows the critical difference diagram on *Accuracy* after applying the null-hypothesis Friedman test and the Wilcoxon-Holm post-hoc test to rank the evaluated methods. The evaluated methods are ranked according to the pairwise comparison of accuracy for every set and bit-width. Then, the average rank across all the data sets and all bit-width settings is computed, as the diagram shows.

Figure 13 shows that our proposals LightTS and AED-LOO achieve the best results. They are clustered in the first rank, and they are statistically more accurate than the other methods. In particular,

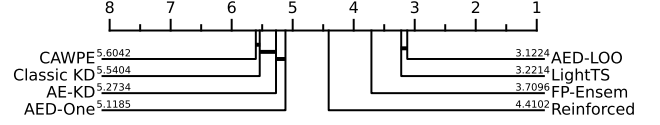


Figure 13: Accuracy Ranking, Full UCR Archive.

LightTS and AED-LOO are statistically more accurate than the full-precision ensemble FP-Ensem, which is ranked in second. This may sound counter-intuitive, but this is possible because knowledge distillation can boost student performance, enabling a well-distilled quantized student to outperform FP-Ensem. Then comes Reinforced, which is followed by a statistically similar cluster of the remaining baselines led by AED-One. Disaggregated results for 4-bit, 8-bit, and 16-bit are shown in Figures 14, 15, and 16, respectively.

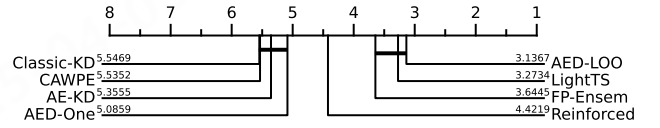


Figure 14: 4-bit Accuracy Ranking.

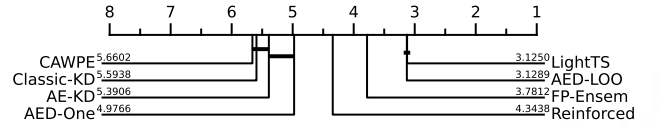


Figure 15: 8-bit Accuracy Ranking.

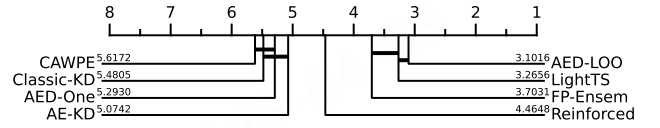


Figure 16: 16-bit Accuracy Ranking.

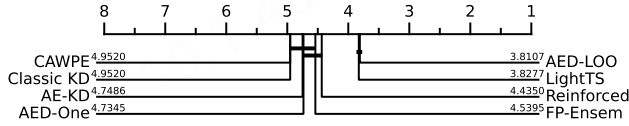
For data sets with 2 or 3 classes, representing 46% of the UCR data sets, we explore the results in Figure 17. It shows a similar ranking as in the overall results, with all the methods having closer results, which is expected as several pairwise comparisons are tied giving the reduced number of classes. This suggests that LightTS works equally well on data sets with few classes.

Detailed results for challenging data sets with many labels are shown in Table 2. For those cases, in addition to the baselines, we consider a full-precision student, denoted as FP-Stud. This is a student model with 32-bit model parameters, which based on knowledge distilled from the full precision ensemble using AED. FP-Stud is supposed to offer an accuracy upper bound to the quantized student with 4-bit, 8-bit, and 16-bit model parameters, denoted by a rectangle, e.g., the fourth row of Table 2.

The main observation is the leading performance in terms of *Accuracy* and *Top-5 Accuracy* of LightTS on all data sets. The metrics for LightTS are consistently better on all data sets, and they are close to the accuracy of FP-Ensem. On data sets such as *Adiac* and *PigArt*, some of the quantized models are able to outperform FP-Ensem, which can be explained by the two sources of training in

**Table 2: Accuracy of Lightweight Student Models, using an Ensemble of InceptionTime Base Models.**

|                  | Accuracy      |      |      | Top-5 Accuracy |      |      | Accuracy      |      |      | Top-5 Accuracy |      |      | Accuracy      |      |      | Top-5 Accuracy |      |      |
|------------------|---------------|------|------|----------------|------|------|---------------|------|------|----------------|------|------|---------------|------|------|----------------|------|------|
| Bit-width        | 4             | 8    | 16   | 4              | 8    | 16   | 4             | 8    | 16   | 4              | 8    | 16   | 4             | 8    | 16   | 4              | 8    | 16   |
|                  | Adiac         |      |      |                |      |      | NonInvECG1    |      |      |                |      |      | PigAirway     |      |      |                |      |      |
| FP-Ensem/FP-Stud | 0.79 / [0.83] |      |      | 0.95 / [0.97]  |      |      | 0.96 / [0.96] |      |      | 1.00 / [1.00]  |      |      | 0.56 / [0.59] |      |      | 0.91 / [0.93]  |      |      |
| Classic KD       | 0.29          | 0.41 | 0.48 | 0.65           | 0.80 | 0.83 | 0.55          | 0.68 | 0.78 | 0.91           | 0.94 | 0.99 | 0.18          | 0.24 | 0.27 | 0.32           | 0.50 | 0.63 |
| AE-KD            | 0.29          | 0.36 | 0.38 | 0.51           | 0.56 | 0.74 | 0.57          | 0.63 | 0.78 | 0.98           | 0.98 | 0.99 | 0.19          | 0.22 | 0.24 | 0.42           | 0.55 | 0.55 |
| Reinforced       | 0.20          | 0.23 | 0.40 | 0.41           | 0.51 | 0.87 | 0.56          | 0.62 | 0.78 | 0.92           | 0.95 | 0.99 | 0.14          | 0.21 | 0.26 | 0.54           | 0.57 | 0.59 |
| CAWPE            | 0.23          | 0.27 | 0.38 | 0.48           | 0.56 | 0.82 | 0.72          | 0.77 | 0.77 | 0.97           | 0.99 | 0.99 | 0.17          | 0.21 | 0.26 | 0.44           | 0.48 | 0.68 |
| AED-LOO          | 0.76          | 0.78 | 0.78 | 0.96           | 0.97 | 0.97 | 0.94          | 0.95 | 0.94 | 1.00           | 1.00 | 1.00 | 0.56          | 0.57 | 0.59 | 0.91           | 0.91 | 0.92 |
| LightTS          | 0.77          | 0.77 | 0.79 | 0.97           | 0.97 | 0.97 | 0.92          | 0.94 | 0.95 | 1.00           | 1.00 | 1.00 | 0.53          | 0.54 | 0.55 | 0.90           | 0.90 | 0.90 |
|                  | Crop          |      |      |                |      |      | NonInvECG2    |      |      |                |      |      | PigArt        |      |      |                |      |      |
| FP-Ensem/FP-Stud | 0.76 / [0.76] |      |      | 0.95 / [0.97]  |      |      | 0.96 / [0.96] |      |      | 1.00 / [1.00]  |      |      | 0.99 / [1.00] |      |      | 0.99 / [1.00]  |      |      |
| Classic KD       | 0.67          | 0.68 | 0.69 | 0.96           | 0.96 | 0.96 | 0.80          | 0.81 | 0.83 | 0.99           | 0.99 | 0.99 | 0.53          | 0.63 | 0.67 | 0.64           | 0.89 | 0.99 |
| AE-KD            | 0.68          | 0.69 | 0.69 | 0.96           | 0.96 | 0.96 | 0.55          | 0.61 | 0.82 | 0.88           | 0.93 | 0.99 | 0.27          | 0.69 | 0.73 | 0.62           | 0.78 | 0.94 |
| Reinforced       | 0.69          | 0.70 | 0.71 | 0.96           | 0.96 | 0.96 | 0.79          | 0.81 | 0.82 | 0.99           | 0.99 | 0.99 | 0.18          | 0.24 | 0.28 | 0.54           | 0.65 | 0.74 |
| CAWPE            | 0.68          | 0.69 | 0.70 | 0.96           | 0.96 | 0.96 | 0.74          | 0.78 | 0.81 | 0.99           | 0.99 | 0.99 | 0.24          | 0.42 | 0.43 | 0.40           | 0.57 | 0.76 |
| AED-LOO          | 0.71          | 0.72 | 0.73 | 0.96           | 0.97 | 0.96 | 0.94          | 0.95 | 0.95 | 1.00           | 1.00 | 1.00 | 0.96          | 0.98 | 0.98 | 1.00           | 1.00 | 1.00 |
| LightTS          | 0.72          | 0.73 | 0.73 | 0.97           | 0.97 | 0.97 | 0.94          | 0.94 | 0.95 | 1.00           | 1.00 | 1.00 | 0.99          | 1.00 | 1.00 | 1.00           | 1.00 | 1.00 |
|                  | FaceAll       |      |      |                |      |      | Phoneme       |      |      |                |      |      | UWave         |      |      |                |      |      |
| FP-Ensem/FP-Stud | 0.81 / [0.85] |      |      | 0.97 / [0.99]  |      |      | 0.29 / [0.30] |      |      | 0.61 / [0.64]  |      |      | 0.93 / [0.95] |      |      | 1.00 / [1.00]  |      |      |
| Classic KD       | 0.75          | 0.77 | 0.80 | 0.84           | 0.88 | 0.89 | 0.26          | 0.27 | 0.29 | 0.59           | 0.61 | 0.62 | 0.75          | 0.79 | 0.81 | 1.00           | 1.00 | 1.00 |
| AE-KD            | 0.74          | 0.76 | 0.75 | 0.83           | 0.88 | 0.94 | 0.24          | 0.23 | 0.26 | 0.56           | 0.57 | 0.58 | 0.77          | 0.79 | 0.81 | 1.00           | 1.00 | 1.00 |
| Reinforced       | 0.68          | 0.72 | 0.79 | 0.85           | 0.86 | 0.86 | 0.26          | 0.28 | 0.29 | 0.60           | 0.60 | 0.62 | 0.72          | 0.72 | 0.80 | 1.00           | 0.99 | 0.99 |
| CAWPE            | 0.69          | 0.70 | 0.76 | 0.83           | 0.85 | 0.86 | 0.22          | 0.23 | 0.26 | 0.58           | 0.61 | 0.63 | 0.66          | 0.79 | 0.81 | 1.00           | 0.99 | 0.99 |
| AED-LOO          | 0.81          | 0.83 | 0.82 | 0.99           | 0.98 | 0.98 | 0.27          | 0.26 | 0.26 | 0.61           | 0.60 | 0.60 | 0.86          | 0.87 | 0.87 | 1.00           | 1.00 | 1.00 |
| LightTS          | 0.81          | 0.82 | 0.84 | 0.98           | 0.98 | 0.98 | 0.26          | 0.27 | 0.27 | 0.61           | 0.62 | 0.63 | 0.87          | 0.88 | 0.89 | 1.00           | 1.00 | 1.00 |



**Figure 17: Accuracy Ranking, 59 Data Sets with 2 or 3 Classes.**

the quantized student, i.e., distillation loss and the classification loss. However, even when quantized models outperform the ensemble, they still perform below the upper-bound performance of FP-Stud. The condition is explained by the fact that 32-bit provides more information, so that model captures better the complete parameters representation, without any precision loss. Therefore, it is expected that it outperforms all quantization settings since they lose some level of precision.

The AED-LOO baseline achieves very good performance for *Accuracy* and *Top-5 Accuracy*. In most of the cases, it is second best. This is expected since AED-LOO shares the AED component with LightTS. Also, the differences between the two methods are related to how they remove teachers. LightTS can evaluate all the  $N$  possible ensembles, regardless the changes in accuracy when some teachers are removed, while AED-LOO stops when the accuracy drops because of its factorial growth in the number of evaluations.

The rather reduced performance of the remaining baselines is related to the premises on which they are built. In all cases, they estimate the weights for the classes probabilities and combine them in a single ensemble, instead of considering independently each base model contribution to the student. This works well in full precision models, but it conflicts with the quantization because the

final ensemble does not provide enough support for the compressed student in contrast to the independent distillation for each member.

In addition, the evaluation on the *UWave* shows the model applicability in multi-dimensional time series, so the compression process it is not constrained by the input dimensions. Also, the small number of labels in the data set, only eight, gives the perfect score for the *Top-5 Accuracy* metric for all the baselines.

**Teacher removal:** To assess the effect of using the Gumbel-Softmax based teacher removal strategy, we compare it with two variants: using the Softmax function to remove teachers and no removing teachers from the ensemble. The results are shown in Table 3 for the *Adiac* data set. Other data sets show similar results. Using the Gumbel-Softmax yields 16–24% improvement in the *Accuracy* and around 5–13% improvement in the *Top-5 Accuracy* with respect to Softmax removal and no removal. Thus, the choice of the Gumbel-Softmax for disregarding base models is justified.

**Table 3: Teacher Removal Strategies for *Adiac* Data Set.**

| Bit-width  | Accuracy    |             |             | Top-5 Accuracy |             |             |
|------------|-------------|-------------|-------------|----------------|-------------|-------------|
|            | 4-bit       | 8-bit       | 16-bit      | 4-bit          | 8-bit       | 16-bit      |
| No removal | 0.55        | 0.61        | 0.64        | 0.86           | 0.92        | 0.93        |
| Softmax    | 0.53        | 0.57        | 0.63        | 0.84           | 0.85        | 0.92        |
| Gumble     | <b>0.77</b> | <b>0.77</b> | <b>0.79</b> | <b>0.97</b>    | <b>0.97</b> | <b>0.97</b> |

**Running time:** We cover training time and inference time separately. The training time is evaluated using the complete UCR archive in Figure 18(a) for all bit settings also considering the null-hypothesis Friedman and the Wilcoxon-Holm post-hoc tests. The

ranking places Classic KD and AED-One in the first group. This is expected, as they share the same complexity as shown by the complexity analysis in Section 3.2.1. Reinforced and LightTS are second, while CAWPE and AE-KD are third, and AED-LOO is last, as it uses an inefficient leave-one-out strategy to remove teachers. The overall results for LightTS show an appropriate trade-off between accuracy and training time. It achieves the best accuracy with a very competitive running time, and the efficiency contenders (Reinforced, AED-One, and Classic KD) are in the bottom segment in terms of overall accuracy. In addition, the training time on the full UCR archive is shown in Figure 18(b) using box plots. Next, inference occurs online, where we use the quantized model to perform classification. Its running time depends only on its size (e.g., 4, 8, or 16 bits) and is independent of the distillation method used. To conclude, since training occurs offline and thus is often not time critical, LightTS is the best choice as it offers the best accuracy. If the training time is critical, AED-One is the best choice as it is as efficient as Classic KD, but is more accurate.

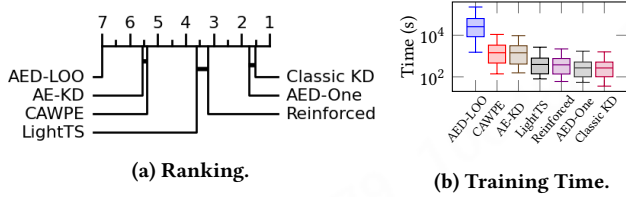


Figure 18: Total Training Time, Full UCR Archive.

**Non-deep Learning Based Models:** To verify that LightTS is a generic framework, which can also apply to base models other than InceptionTime, we consider non-deep learning based models, *TDE*, *CIF*, and *Forest*, Table 4 shows results on data sets *Adiac* and *PigAirway*, due to the space limitation. Results on the remaining data sets exhibit similar trends.

The main observation is that LightTS offers much better *Accuracy* and *Top-5 Accuracy* performance than the baselines. Specifically for *Accuracy*, the level of improvement with respect to the baselines is, for most of the cases, around a factor of three. The reason for the large difference is the adaptability of LightTS when choosing teachers. It is able to identify the candidates that are better aligned with the student settings and distills knowledge from them, while the other methods include all the base models regardless of their performance.

Next, in all cases, the full-precision ensemble outperforms the distilled cases at *Accuracy*, meaning that there is still room for improvement of the quantized student models. As the differences between the non-deep learning base models and LightTS are higher than when using InceptionTime as base models, it appears that the architectural differences between the teachers and the student have some effect on the distillation performance.

**Hyper-parameter sensitivity:** In Figure 19, the sensitivity for the hyper-parameters  $\alpha$  and  $\tau$  is evaluated for the *Adiac* data set in a configuration of 4 bits. It shows that  $\alpha$  is stable when the two losses are balanced. Then, the accuracy changes are steeper with  $\tau$  modifications, meaning that it leads to a different selection of teachers, affecting the overall performance. The choice of 0.5 for

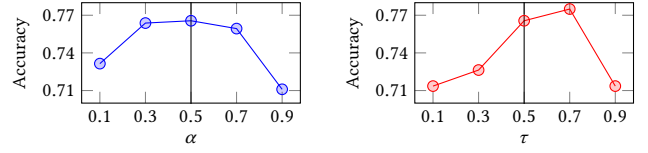


Figure 19: Hyper-parameter Sensitivity, *Adiac*, 4-bit.

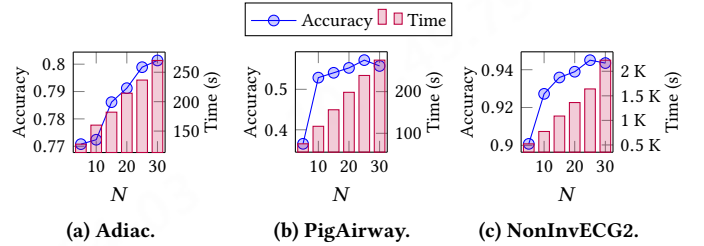


Figure 20: Effect of the Number of Base Models  $N$ .

both parameters seems precise, since it is among the best possible options. Other data sets and bit settings show similar trends.

**Effect of the number of base models  $N$ :** Figure 20 shows the impact of the number of base models  $N$  on *Accuracy* and *Training time* for data sets *Adiac*, *PigAirway*, and *NonInvECG2*. In all cases, when  $N$  is small, the accuracy results are significantly affected mainly because the opportunities to choose appropriate base models and to disregard unsuitable base models are reduced. Then, as  $N$  increases, the accuracy is relatively stable within a range of 5%, achieving the best accuracy with some 25 or 30 base models. For *PigAirway* and *NonInvECG2*, the results decrease slightly when the number of base models reaches 30, which we attribute to the added uncertainty caused by having more base models, i.e., removing base models becomes increasingly arbitrary as more options are available. The changes for *Adiac* are very slightly after considering 25 models, suggesting it will reach a maximum closer to that range. The total training time grows linearly as the number of base models increases, which is consistent with the complexity analysis.

**4.2.2 Problem Scenario 2.** To find optimal student settings using the proposed encoded multi-objective Bayesian optimization, we initialize the process with  $P = 10$  random settings and search for 40 additional settings, meaning that  $Q = 50$ . We show the results on the data set *Adiac* given the space limitation. Experiments conducted in other data sets show similar observations.

**Feature transformation effect:** To evaluate the effect of using the two-phase encoder, we assess it in comparison to (1) a single phase encoder using the reconstruction loss only (i.e., an autoencoder) but without considering the accuracy adjustment loss function, (2) using the original space where the values are normalized, and (3) the original discrete space. We use these different spaces to train a GP accuracy estimator and evaluate the estimated accuracy vs. ground truth accuracy of 50 randomly sampled settings. The results for data sets *Adiac*, *PigAirway*, and *NonInvECG2* in Table 5 show that the proposed two-phase encoder gives the most accurate

**Table 4: Accuracy of Lightweight Students, using Ensembles with Base Models other than InceptionTime.**

|           |                  | TDE           |             |             |                |             |             | CIF           |             |             |                |             |             | Forest        |             |             |                |             |             |
|-----------|------------------|---------------|-------------|-------------|----------------|-------------|-------------|---------------|-------------|-------------|----------------|-------------|-------------|---------------|-------------|-------------|----------------|-------------|-------------|
|           |                  | Accuracy      |             |             | Top-5 Accuracy |             |             | Accuracy      |             |             | Top-5 Accuracy |             |             | Accuracy      |             |             | Top-5 Accuracy |             |             |
| Bit-width |                  | 4             | 8           | 16          | 4              | 8           | 16          | 4             | 8           | 16          | 4              | 8           | 16          | 4             | 8           | 16          | 4              | 8           | 16          |
| Adiac     | FP-Ensem/FP-Stud | 0.50 / [0.61] |             |             | 0.79 / [0.83]  |             |             | 0.79 / [0.84] |             |             | 0.97 / [0.97]  |             |             | 0.74 / [0.77] |             |             | 0.95 / [0.96]  |             |             |
|           | Classic KD       | 0.14          | 0.13        | 0.12        | 0.37           | 0.32        | 0.31        | 0.24          | 0.27        | 0.24        | 0.48           | 0.43        | 0.56        | 0.21          | 0.23        | 0.18        | 0.57           | 0.42        | 0.40        |
|           | AE-KD            | 0.13          | 0.18        | 0.16        | 0.35           | 0.41        | 0.37        | 0.21          | 0.24        | 0.21        | 0.46           | 0.43        | 0.47        | 0.23          | 0.29        | 0.21        | 0.55           | 0.71        | 0.49        |
|           | Reinforced       | 0.11          | 0.13        | 0.14        | 0.30           | 0.31        | 0.30        | 0.24          | 0.19        | 0.19        | 0.53           | 0.37        | 0.37        | 0.15          | 0.18        | 0.19        | 0.41           | 0.36        | 0.39        |
|           | CAWPE            | 0.12          | 0.15        | 0.14        | 0.34           | 0.35        | 0.33        | 0.15          | 0.14        | 0.14        | 0.35           | 0.31        | 0.34        | 0.12          | 0.15        | 0.14        | 0.34           | 0.35        | 0.33        |
|           | LightTS          | <b>0.32</b>   | <b>0.37</b> | <b>0.41</b> | <b>0.65</b>    | <b>0.69</b> | <b>0.75</b> | <b>0.66</b>   | <b>0.71</b> | <b>0.71</b> | <b>0.93</b>    | <b>0.94</b> | <b>0.95</b> | <b>0.68</b>   | <b>0.69</b> | <b>0.71</b> | <b>0.93</b>    | <b>0.93</b> | <b>0.95</b> |
| PigAirway | FP-Ensem/FP-Stud | 0.76 / [0.80] |             |             | 0.97 / [0.98]  |             |             | 0.42 / [0.46] |             |             | 0.93 / [0.94]  |             |             | 0.42 / [0.45] |             |             | 0.94 / [0.95]  |             |             |
|           | Classic KD       | 0.22          | 0.23        | 0.14        | 0.63           | 0.63        | 0.42        | 0.11          | 0.13        | 0.11        | 0.33           | 0.39        | 0.34        | 0.14          | 0.15        | 0.10        | 0.40           | 0.43        | 0.34        |
|           | AE-KD            | 0.15          | 0.20        | 0.18        | 0.39           | 0.49        | 0.49        | 0.11          | 0.13        | 0.17        | 0.35           | 0.35        | 0.47        | 0.11          | 0.10        | 0.11        | 0.33           | 0.33        | 0.32        |
|           | Reinforced       | 0.13          | 0.13        | 0.19        | 0.36           | 0.43        | 0.49        | 0.12          | 0.10        | 0.07        | 0.29           | 0.31        | 0.27        | 0.08          | 0.10        | 0.14        | 0.25           | 0.37        | 0.40        |
|           | CAWPE            | 0.11          | 0.12        | 0.12        | 0.33           | 0.38        | 0.35        | 0.11          | 0.12        | 0.12        | 0.32           | 0.35        | 0.36        | 0.11          | 0.12        | 0.12        | 0.33           | 0.38        | 0.35        |
|           | LightTS          | <b>0.60</b>   | <b>0.66</b> | <b>0.70</b> | <b>0.93</b>    | <b>0.95</b> | <b>0.96</b> | <b>0.36</b>   | <b>0.40</b> | <b>0.45</b> | <b>0.68</b>    | <b>0.73</b> | <b>0.77</b> | <b>0.33</b>   | <b>0.38</b> | <b>0.42</b> | <b>0.66</b>    | <b>0.70</b> | <b>0.74</b> |

estimation for the unevaluated settings’ accuracy, which outperforms the one-phase autoencoder. In addition, simply normalizing the values in settings fails to improve accuracy.

**Table 5: Gaussian Processes Accuracy Estimation.**

|                   | Adiac       |             | PigAirway   |             | NonInvECG2  |             |
|-------------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                   | MAE         | MAPE        | MAE         | MAPE        | MAE         | MAPE        |
| Original          | 0.12        | 0.31        | 0.05        | 0.25        | <b>0.04</b> | 0.06        |
| Normalized        | 0.11        | 0.31        | 0.05        | 0.27        | 0.05        | 0.06        |
| Single Encoder    | 0.09        | 0.24        | 0.05        | 0.23        | <b>0.04</b> | <b>0.05</b> |
| Two-phase Encoder | <b>0.08</b> | <b>0.23</b> | <b>0.04</b> | <b>0.16</b> | <b>0.04</b> | <b>0.05</b> |

**Base settings improvement:** In Figure 21(a), 21(b), and 21(c), we show how the encoded Bayesian optimization is able to improve the results of the fixed settings used in the first set of experiments for the *Adiac*, *PigAirway*, and *NonInvECG2* data sets, respectively. In the figure, we include the three lightweight student cases of 4, 8, and 16 bits (c.f. Table 2) denoted as “Base settings,” and denoting upper-left areas for possible improvements, meaning that any point in that area has a better accuracy at a reduced size. We only show the results in the areas for improvement to keep the figures clear, so fewer than  $Q = 50$  points are shown. Then, we illustrate the effect of the optimization with two levels of flexibility. First, we keep the number of layers fixed as the base settings and apply the optimization limiting the search space to only bit-width. In the figure, these are denoted as “Fixed layers.” Then, we explore the complete search space, varying the number of layers, filter length, and bit-width, shown in the figure as “Encoded MOBO.” The evaluation shows that the optimization process is able to find better student settings than with the base settings, i.e., improving accuracy while consuming less space. In addition, the figure shows that exploring only the bit-width space enables better settings, but that it is potentially more beneficial to explore the complete search space, since there are better settings when also the layers can be varied.

**Pareto frontiers:** Using the *Adiac*, *PigAirway*, and *NonInvECG2* data sets, we show in Figures 22(a), 22(b), and 22(c) a comparison of the Pareto frontiers when using different methods. We include the

**Table 6: Optimization Running Time (hours).**

|              | Adiac | PigAirway | NonInvECG2 |
|--------------|-------|-----------|------------|
| Random       | 4.61  | 2.72      | 19.65      |
| MOBO         | 5.78  | 3.45      | 28.77      |
| Encoded MOBO | 5.85  | 3.48      | 29.82      |

proposed encoded Multi-objective Bayesian optimization (“Encoded MOBO”); classic Multi-objective Bayesian optimization (“MOBO”), where the settings are not encoded; and randomly choosing all  $Q$  settings (“Random”). The Encoded MOBO obtains a better Pareto frontier than do the other two methods, i.e., it is closer to the upper, left corner. Thus, the frontier includes settings with relatively small model size and high accuracy, meaning that it is possible to find very competitive settings with very strict memory constraints.

The methods are evaluated in terms of running time, as shown in Table 6. The “Random” search is faster since it does not optimize the search, but that leads to a poor Pareto frontier. Next, the differences in running time between the “MOBO” and “Encoded MOBO” optimizations are relatively small, which is expected because of the similar optimization processes.

To evaluate the effect of initializing the optimization process with evaluated settings sets of different size, i.e., with fixed  $Q$  while changing  $P$ , we build the Pareto frontiers with  $P$  equal to 5, 10, 20, 30, and 40—see Figure 23. Using  $P = 5$  constrains the process substantially, and it only identifies models of relatively large size, implying that a poorly-initialized GP may mislead the optimization process of BO. In contrast, using  $P = 10, 20, 30$  yields similar Pareto frontiers, and we empirically find that  $P = 10$  is a good choice since it successfully initializes the exploration similarly to larger values.

## 5 RELATED WORK

**Time Series Classification:** Time series classification has seen substantial advances, and state-of-the-art methods offer impressive accuracy [18, 44]. However, the state-of-the-art methods require significant computing resources, making them inapplicable in resource-limited settings. Some methods (e.g., [37]) use large ensembles, that combine the results from dozens of single models to compute a result, while other methods (e.g., [12, 13, 22, 32])

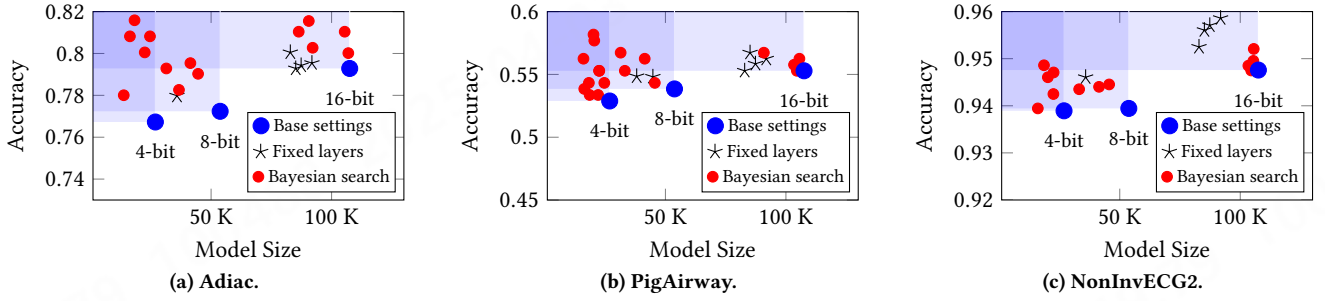


Figure 21: Base Settings Improvement.

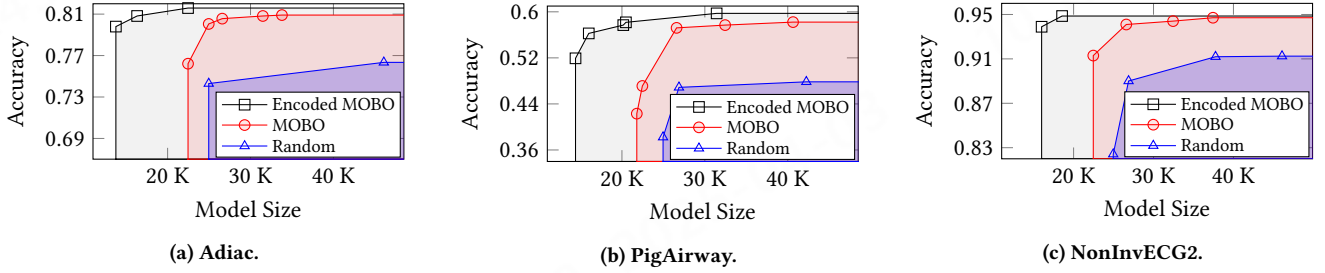


Figure 22: Identifying Pareto Frontiers.

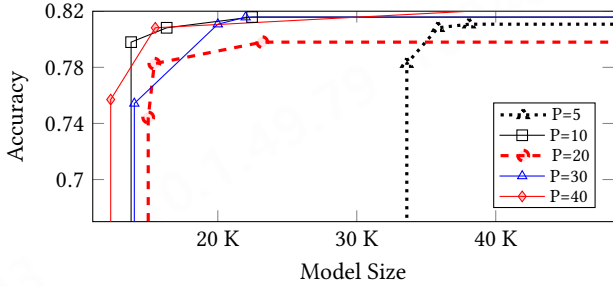


Figure 23: Varying  $P$ , Encoded MOBO, Adiac.

require multiple data transformations for exploring time series properties [4, 26], which requires substantial memory. Then, deep learning [19, 49] and non-deep learning [14, 32, 36, 38, 52] based ensembles are competitive models, so they are considered as inputs for LightTS.

**Knowledge Distillation:** Knowledge distillation has been studied since its introduction [25], with studies considering single teachers [8, 41, 43, 45], ensembles of diverse teachers [17, 21, 54, 56], and techniques that aim to improve performance via self-distillation [55] and online student feedback [7]. When compressing from an ensemble of diverse teachers into a student, existing studies consider mainly a student with full-precision parameters, while we consider lightweight students, e.g., with 4, 8, or 16-bit quantized parameters. This difference calls for a more flexible distillation strategy and means to identify the Pareto optimal frontier, which is not considered by existing studies. Although different methods exist that balance different teachers’ contributions to an ensemble [17, 31, 54], which then guides the knowledge distillation, no studies consider the removal of teachers. We propose a novel bi-level optimization modeling that not only assigns appropriate weights to useful teachers, but also facilitates removal of irrelevant teachers, thus achieving more flexible distillation and higher accuracy, as shown in the experiments.

**Pareto frontier:** Existing studies do not consider how to build a Pareto frontier, partially because they consider only students with full-precision parameters [45, 47, 53]. In our setting, it is possible to use different bit-widths in different layers, making it challenging to identify Pareto frontiers efficiently. To contend with this challenge, we propose a novel encoding scheme along with an Encoded MOBO method to identify a Pareto frontier by only evaluating a small number of student model settings. Skyline querying [5, 51] is able to efficiently identify the Pareto frontier for an input data set  $D$  in the form of (accuracy, size) pairs. However, naively constructing  $D$  requires evaluating the accuracy of large numbers of student settings, which is prohibitively expensive. The Encoded MOBO aims at constructing an appropriate input data set  $D$  by evaluating the accuracy of a small number of student settings, upon which any skyline querying algorithm can be applied. Thus, skyline querying and Encoded MOBO are orthogonal.

## 6 CONCLUSIONS AND FUTURE WORK

This paper proposes LightTS, a novel and flexible framework that extends state-of-the-art time series classification to resource-limited devices. First, LightTS is able to adaptively distil the knowledge from a set of high-capacity, large base models to a lightweight model. In doing this, it employs a bi-level optimization approach to estimate the benefit of each base model during the distillation. Second, an encoded multi-objective Bayesian optimization method is proposed to search for the Pareto optimal settings for lightweight models under varying space constraints. The results of experimental studies offer evidence of the effectiveness of LightTS when used on a variety of real-world time series data sets with different base models.

In future work, it is of interest to explore more flexible distillation paradigms, such as online distillation that does not require pretrained base models. It is also of interest to study how to adapt distillation to streaming settings.

## REFERENCES

- [1] U. Rajendra Acharya, Hamido Fujita, Shu Lih Oh, Yuki Hagiwara, Jen Hong Tan, and Muhammad Adam. 2017. Application of deep convolutional neural network for automated detection of myocardial infarction using ECG signals. *Inf. Sci.* 415 (2017).
- [2] Charu C. Aggarwal and Saket Sathe. 2017. *Outlier Ensembles - An Introduction*.
- [3] Charles Blake, Vitaly Kurin, Maximilian Igl, and Shimon Whiteson. 2021. Snowflake: Scaling GNNs to high-dimensional continuous control via parameter freezing. In *NIPS*.
- [4] Paul Boniol and Themis Palpanas. 2020. Series2Graph: Graph-based Subsequence Anomaly Detection for Time Series. *Proc. VLDB Endow.* 13, 11 (2020).
- [5] Stephan Börzsönyi, Donald Kossmann, and Konrad Stocker. 2001. The Skyline Operator. In *ICDE*.
- [6] David Campos, Tung Kieu, Chenjuan Guo, Feiteng Huang, Kai Zheng, Bin Yang, and Christian S. Jensen. 2021. Unsupervised Time Series Outlier Detection with Diversity-Driven Convolutional Ensembles. *Proc. VLDB Endow.* 15, 3 (2021).
- [7] Defang Chen, Jian-Ping Mei, Can Wang, Yan Feng, and Chun Chen. 2020. Online Knowledge Distillation with Diverse Peers. In *AAAI*.
- [8] Tri Dao, Govinda M. Kamath, Vasilis Syrgkanis, and Lester Mackey. 2021. Knowledge Distillation as Semiparametric Inference. In *ICLR*.
- [9] Hoang Anh Dau, Anthony J. Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn J. Keogh. 2019. The UCR time series archive. *IEEE CAA J. Autom. Sinica* 6, 6 (2019).
- [10] Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. 2020. Differentiable Expected Hypervolume Improvement for Parallel Multi-Objective Bayesian Optimization. In *NIPS*.
- [11] Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. 2021. Parallel Bayesian Optimization of Multiple Noisy Objectives with Expected Hypervolume Improvement. In *NIPS*.
- [12] Angus Dempster, François Petitjean, and Geoffrey I. Webb. 2020. ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. *Data Min. Knowl. Discov.* 34, 5 (2020).
- [13] Angus Dempster, Daniel F. Schmidt, and Geoffrey I. Webb. 2021. MiniRocket: A Very Fast (Almost) Deterministic Transform for Time Series Classification. In *KDD*.
- [14] Houtao Deng, George C. Runger, Eugene Tuv, and Vladimir Martynov. 2013. A time series forest for classification and feature extraction. *Inf. Sci.* 239 (2013).
- [15] Aryan Deshwal and Janardhan Rao Doppa. 2021. Combining Latent Space and Structured Kernels for Bayesian Optimization over Combinatorial Spaces. In *NIPS*.
- [16] Thomas G. Dietterich. 2000. Ensemble Methods in Machine Learning. In *MCS (Lecture Notes in Computer Science)*, Vol. 1857.
- [17] Shangchen Du, Shan You, Xiaojie Li, Jianlong Wu, Fei Wang, Chen Qian, and Changshui Zhang. 2020. Agree to Disagree: Adaptive Ensemble Knowledge Distillation in Gradient Space. In *NIPS*.
- [18] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. 2019. Deep learning for time series classification: a review. *Data Min. Knowl. Discov.* 33, 4 (2019).
- [19] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. 2020. InceptionTime: Finding AlexNet for time series classification. *Data Min. Knowl. Discov.* 34, 6 (2020).
- [20] Milton Friedman. 1940. A Comparison of Alternative Tests of Significance for the Problem of  $m$  Rankings. *Annals of Mathematical Statistics* 11 (1940).
- [21] Takashi Fukuda, Masayuki Suzuki, Gakuto Kurata, Samuel Thomas, Jia Cui, and Bhuvana Ramabhadran. 2017. Efficient Knowledge Distillation from an Ensemble of Teachers. In *Interspeech*.
- [22] Anna Gogolou, Theophanis Tsandilas, Karima Echihabi, Anastasia Bezerianos, and Themis Palpanas. 2020. Data Series Progressive Similarity Search with Probabilistic Quality Guarantees. In *SIGMOD*.
- [23] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. 2019. Differentiable Soft Quantization: Bridging Full-Precision and Low-Bit Neural Networks. In *ICCV*.
- [24] Emil Gumbel. 1955. Statistical theory of extreme values and some practical applications: a series of lectures. *Journal of the Royal Statistical Society* 118, 1 (1955).
- [25] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. 2015. Distilling the Knowledge in a Neural Network. In *NIPS*.
- [26] Van Long Ho, Nguyen Ho, and Torben Bach Pedersen. 2021. Efficient Temporal Pattern Mining in Big Time Series Using Mutual Information. *Proc. VLDB Endow.* 15, 3 (2021).
- [27] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979).
- [28] Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical Reparameterization with Gumbel-Softmax. In *ICLR*.
- [29] Donald R. Jones, Matthias Schonlau, and William J. Welch. 1998. Efficient Global Optimization of Expensive Black-Box Functions. *J. Glob. Optim.* 13, 4 (1998).
- [30] Fazle Karim, Somshubra Majumdar, and Houshang Darabi. 2019. Insights Into LSTM Fully Convolutional Networks for Time Series Classification. *IEEE Access* 7 (2019).
- [31] James Large, Jason Lines, and Anthony J. Bagnall. 2019. A probabilistic classifier ensemble weighting scheme based on cross-validated accuracy estimates. *Data Min. Knowl. Discov.* 33, 6 (2019).
- [32] Carl Henning Lubba, Sarab S. Sethi, Philip Knaute, Simon R. Schultz, Ben D. Fulcher, and Nick S. Jones. 2019. catch22: CAnonical Time-series CHaracteristics - Selected through highly comparative time-series analysis. *Data Min. Knowl. Discov.* 33, 6 (2019).
- [33] Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard S. Zemel. 2016. Understanding the Effective Receptive Field in Deep Convolutional Neural Networks. In *NIPS*.
- [34] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. 2017. The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables. In *ICLR*.
- [35] Chris J. Maddison, Daniel Tarlow, and Tom Minka. 2014. A\* Sampling. In *NIPS*.
- [36] Matthew Middlehurst, James Large, and Anthony J. Bagnall. 2020. The Canonical Interval Forest (CIF) Classifier for Time Series Classification. In *BigData*.
- [37] Matthew Middlehurst, James Large, Michael Flynn, Jason Lines, Aaron Bostrom, and Anthony J. Bagnall. 2021. HIVE-COTE 2.0: a new meta ensemble for time series classification. *Mach. Learn.* 110, 11 (2021).
- [38] Matthew Middlehurst, William Vickers, and Anthony J. Bagnall. 2019. Scalable Dictionary Classifiers for Time Series Classification. In *IDEAL*, Vol. 11871.
- [39] Sayan Mukherjee, Partha Niyogi, Tomaso A. Poggio, and Ryan M. Rifkin. 2006. Learning theory: stability is sufficient for generalization and necessary and sufficient for consistency of empirical risk minimization. *Adv. Comput. Math.* 25, 1-3 (2006).
- [40] James O'Neill. 2020. An Overview of Neural Network Compression. *CoRR* abs/2006.03669 (2020).
- [41] Antonio Polino, Razvan Pascanu, and Dan Alistarh. 2018. Model compression via distillation and quantization. In *ICLR*.
- [42] Carl Edward Rasmussen and Christopher K. I. Williams. 2004. *Gaussian Processes for Machine Learning*.
- [43] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. 2015. FitNets: Hints for Thin Deep Nets. In *ICLR*.
- [44] Alejandro Pasos Ruiz, Michael Flynn, James Large, Matthew Middlehurst, and Anthony J. Bagnall. 2021. The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Min. Knowl. Discov.* 35, 2 (2021).
- [45] Yonglong Tian, Dilip Krishnan, and Phillip Isola. 2020. Contrastive Representation Distillation. In *ICLR*.
- [46] Austin Tripp, Erik A. Daxberger, and José Miguel Hernández-Lobato. 2020. Sample-Efficient Optimization in the Latent Space of Deep Generative Models via Weighted Retraining. In *NIPS*.
- [47] Frederick Tung and Greg Mori. 2018. CLIP-Q: Deep Network Compression Learning by In-Parallel Pruning-Quantization. In *CVPR*.
- [48] Jindong Wang, Yiqiang Chen, Shuji Hao, Xiaohui Peng, and Lisha Hu. 2019. Deep learning for sensor-based activity recognition: A survey. *Pattern Recognit. Lett.* 119 (2019).
- [49] Zhiguang Wang, Weizhong Yan, and Tim Oates. 2017. Time series classification from scratch with deep neural networks: A strong baseline. In *IJCNN*.
- [50] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945).
- [51] Bin Yang, Chenjuan Guo, Christian S. Jensen, Manohar Kaul, and Shuo Shang. 2014. Stochastic skyline route planning under time-varying uncertainty. In *ICDE*.
- [52] Ze Yang, Linjun Shou, Ming Gong, Wutao Lin, and Daxin Jiang. 2020. Model Compression with Two-stage Multi-teacher Knowledge Distillation for Web Question Answering System. In *WSDM*.
- [53] Junho Yim, Donggyu Joo, Ji-Hoon Bae, and Junmo Kim. 2017. A Gift from Knowledge Distillation: Fast Optimization, Network Minimization and Transfer Learning. In *CVPR*.
- [54] Fei Yuan, Linjun Shou, Jian Pei, Wutao Lin, Ming Gong, Yan Fu, and Daxin Jiang. 2021. Reinforced Multi-Teacher Selection for Knowledge Distillation. In *AAAI*.
- [55] Linfeng Zhang, Chenglong Bao, and Kaisheng Ma. 2022. Self-Distillation: Towards Efficient and Compact Neural Networks. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 8 (2022).
- [56] Wentao Zhang, Jiawei Jiang, Yingxia Shao, and Bin Cui. 2020. Efficient Diversity-Driven Ensemble for Deep Neural Networks. In *ICDE*.
- [57] Xuchao Zhang, Yifeng Gao, Jessica Lin, and Chang-Tien Lu. 2020. TapNet: Multivariate Time Series Classification with Attentional Prototypical Network. In *AAAI*.
- [58] Shuai Zhao, Frede Blaabjerg, and Huai Wang. 2021. An Overview of Artificial Intelligence Applications for Power Electronics. *IEEE Transactions on Power Electronics* 36, 4 (2021).
- [59] Zhi-Hua Zhou, Jianxin Wu, and Wei Tang. 2002. Ensembling neural networks: Many could be better than all. *Artif. Intell.* 137, 1-2 (2002).